

RFC: Shared Chunk Cache Design

Clay Carper clay.carper@lifeboat.llc
John Mainzer john.mainzer@lifeboat.llc

This document outlines the design of the Shared Chunk Cache (SCC). Initially, SCC will support I/O operations for structured chunks—a newly introduced HDF5 storage paradigm for sparse and variable-length data. In the future, it may replace the current chunk cache implementation, extending its functionality to support dense chunks as well. The cache is also designed to enable multi-threaded access in the future.

This document describes the Version 1 design and current implementation of the Shared Chunk Cache. It may be updated as future SCC functionality is developed.

1	Introduction	4
1.1.	Current Implementation Status	4
2	Overview of Internal Data Types and Structures	5
2.1.	H5SC_tag_t – SCC Operation/Debugging Tags	7
2.2.	H5SC_size_est_source_t – Resident-Size Estimate Source.....	9
2.3.	H5SC_chunk_key_t – Chunk Indexing	10
2.4.	H5SC_chunk_t – Individual Cache Entry Structure	11
2.5.	H5SC_io_scratch_t – Structure Used for Dynamically Sized Scratch Storage.....	13
2.6.	H5SC_io_info_t – Structure Used for an I/O Request on a Dataset	17
2.7.	H5SC_io_sel_chunk_t – Callback-Related Chunk Information for I/O Ops.....	18
2.8.	H5SC_dset_header_t – Dataset-Based Chunk Tracking	21
2.9.	H5SC_stats_t – Per-File Global Cache Statistics	24
2.10.	H5SC__cache_config_t – Shared Chunk Cache Configuration.....	27
2.11.	H5SC_t – Per-File Global Cache Management.....	28
3	Theory of Operations	30
3.1.	File Open.....	30
3.2.	Dataset Open	31
3.3.	Dataset Flush	31
3.4.	Dataset Erase.....	32
3.5.	Applying Filters Through the SCC.....	33
3.6.	Dataset Change Extent	34
3.7.	Dataset Close	35
3.8.	Writing Data Through the SCC.....	36
3.9.	Reading Data Through the SCC	37
3.10.	Flush the Cache.....	37
3.11.	File Close.....	38
4	Detailed Component Discussions	38
4.1.	Chunk Tracking Through the Hash Table	38
4.2.	Eviction	38
4.2.1	Additional Eviction Strategies.....	39
4.3.	Batching.....	40
4.4.	Public Configuration Structure	41

4.5.	Filter Configuration and Usage	41
4.5.1	GZIP / DEFLATE Filtering	42
4.5.2	SZIP Filtering	42
4.5.3	Mixed Filter Pipelines	43
5	Top Level API	44
6	SCC Internal Functions	48
6.1.	Hash table specific functions	48
6.2.	DLL specific functions	49
6.3.	H5SC_chunk_t specific functions	50
6.4.	I/O specific functions	50
6.5.	Eviction specific functions	53
6.6.	Size accounting functions	55
6.7.	Extensible dimension-related functions	57
6.8.	Statistics-specific functions	58
7	Layout Callbacks	61
7.1.1	Callback struct	68
8	Call Tree Diagrams	68
8.1.	Reading data through H5Dread():	68
8.1.1	Data Read Call Tree by Subsystem:	70
8.2.	Writing data through H5Dwrite():	70
8.2.1	Data Write Call Tree by Subsystem:	72
8.3.	Supporting Initialization and Dataset Setup	73
8.4.	Eviction and Flush Subsystems	73
8.5.	H5Derase() Functionality	74
8.6.	H5Dset_extent Functionality	76
9	Test Coverage Summary	78
10	Multi-threaded support	79
10.1.	Multi-threaded operations in SCC	79
10.2.	Additional code for supporting multi-thread operations in SCC	79
11	Acknowledgement	80
12	Appendix A: SCC Version 0 Implementation Description	80
13	References	82

1 Introduction

This document outlines the design for the Shared Chunk Cache (SCC). The new cache not only provides support for structured chunks, a new HDF5 storage layout, but also provides an opportunity to address additional requirements [1] and known limitations of the existing chunk cache. For example, a single cache will be utilized across all datasets in a single HDF5 file, enabling the SCC to utilize a minimalistic hash table, while also utilizing portions of memory that would have otherwise been inaccessible due to the nature of the previously utilized cache-per-dataset approach. The remainder of this document will focus on the design of the SCC, including the proposed structures, how they satisfy the design requirements, and how various procedures will utilize these components.

1.1. Current Implementation Status

The Shared Chunk Cache is currently implemented as a per-file cache used by structured-chunk datasets. The implementation supports creation and destruction of the per-file SCC, dataset registration and removal, cache-wide chunk lookup, per-dataset and cache-wide LRU tracking, resident chunk reuse, batched read and write processing, cache-size enforcement, flushing and eviction, sparse erase operations, defined-value queries, dataset extent changes, and structured-chunk filter pipelines. The current implementation supports both clean and dirty chunk reclamation. Cache policy distinguishes active I/O pressure from quiescent retention: the active limit controls memory admission while an I/O operation is being processed, while the quiescent limit controls the amount of resident data retained between operations. Pinned chunks are protected from reclamation, and dataset minimum-retention targets apply to quiescent trimming rather than active-pressure reclamation.

Structured-chunk operations through the SCC currently support `H5S_SEL_ALL` and `H5S_SEL_HYPERSLABS` file-space selections. Empty selections are accepted as no-op operations. `H5S_SEL_POINTS` selections are intentionally unsupported by the current SCC interface and are rejected. This selection restriction applies to SCC selection decomposition used by read/write processing, erase operations, and defined-value queries. Point-selection support may be added in a future implementation. Structured-chunk I/O currently supports SCC-managed sparse chunks, including filtered chunks, partial-edge chunks, extensible datasets, erase operations, and persistence across file close/reopen. The generic SCC layout interface defines optional selection-I/O and vector-I/O callbacks. Structured chunks currently provide implementations of `vector_read` and `vector_write`, but these callbacks are not integrated into the normal SCC read/write execution paths and are presently exercised as proof-of-concept functionality in dedicated callback tests. `selection_read` and `selection_write` are not currently implemented for structured chunks.

Current limitations and future work include multi-threaded SCC operation, integration of the selection/vector direct-I/O callback paths, point-selection support, and possible use of the SCC for traditional dense chunk storage. These remain design goals rather than currently supported functionality. Additional optimization and performance characterization of batching, resident-size estimation, and cache policy may continue without changing the fundamental SCC architecture described here.

2 Overview of Internal Data Types and Structures

To support and inform the underlying design decisions, this document will first focus on the underlying data structures which have been developed for the initial single-thread implementation of the SCC. Additionally, considerations for future multi-thread support outlined in section 10 have been influential in this development. Further, the diagram below provides an illustration of the structures discussed in this document.



2.1. H5SC_tag_t – SCC Operation/Debugging Tags

```

/*****
*
* Enumeration: H5SC_tag_t
*
* Info
*
* The enumeration defines tags used to identify the most recent internal
* SCC operation performed on an H5SC_chunk_t or H5SC_dset_header_t
* structure. These values are primarily used for debugging, validation,
* and tracking state transitions during hash table, LRU, I/O, flush, and
* eviction operations.
*
* An H5SC_tag_t value is stored in the last_op field of both
* H5SC_chunk_t and H5SC_dset_header_t. The structure containing the tag
* provides the context necessary to interpret operations that are common
* to both structure types. For example, H5SC_TAG_LRU_INSERT stored in an
* H5SC_chunk_t refers to insertion into a dataset's chunk LRU list, while
* the same tag stored in an H5SC_dset_header_t refers to insertion into
* the SCC dataset LRU list.
*
* Tags are therefore operation-oriented rather than structure-oriented.
* A separate tag is not required for equivalent operations performed on
* chunks and dataset headers. Some tags, particularly those associated
* with pinning and unpinning during I/O, apply only to H5SC_chunk_t
* structures.
*
* Values
*
* H5SC_TAG_NONE - Indicates that no SCC operation has been associated
* with the structure. This value is used as an uninitialized or default
* tag where appropriate.
*
* H5SC_TAG_CREATE - Indicates that the H5SC_chunk_t or H5SC_dset_header_t
* structure was created.
*
* H5SC_TAG_HT_INSERT - Indicates that the structure was inserted into its
* associated SCC hash table. For H5SC_chunk_t, this refers to the chunk
* hash table. For H5SC_dset_header_t, this refers to the dataset hash
* table.
*
* H5SC_TAG_HT_DELETE - Indicates that the structure was removed from its
* associated SCC hash table.
*
* H5SC_TAG_LRU_INSERT - Indicates that the structure was inserted into its
* associated LRU list. For H5SC_chunk_t, this refers to the associated
* dataset's chunk LRU list. For H5SC_dset_header_t, this refers to the
* SCC dataset LRU list.
*
* H5SC_TAG_LRU_REMOVE - Indicates that the structure was removed from its
* associated LRU list.
*
* H5SC_TAG_LRU_PROMOTE - Indicates that the structure was promoted within
* its associated LRU list to reflect recent use.
*
*****/

```

```

*   H5SC_TAG_LRU_TOUCH - Indicates that the recency of the structure was
*   updated as the result of an access or other operation affecting its
*   position within an LRU list.
*
*   H5SC_TAG_FLUSH_DIRTY - Indicates that the structure was involved in an
*   operation responsible for flushing dirty cached chunk data.
*
*   H5SC_TAG_FLUSH - Indicates that the structure was involved in a general
*   SCC flush operation.
*
*   H5SC_TAG_EVICT - Indicates that the structure was involved in an SCC
*   eviction operation.
*
*   H5SC_TAG_LOOKUP - Indicates that the structure was accessed during an
*   SCC lookup operation.
*
*   H5SC_TAG_PIN_IOINIT - Indicates that a chunk was pinned while
*   initializing an I/O request. This tag applies to H5SC_chunk_t
*   structures.
*
*   H5SC_TAG_PIN_IOINIT_CACHED - Indicates that an existing cached chunk
*   was pinned while initializing an I/O request. This tag applies to
*   H5SC_chunk_t structures.
*
*   H5SC_TAG_UNPIN_WRITE_DONE - Indicates that a chunk was unpinned after
*   successful completion of a write operation. This tag applies to
*   H5SC_chunk_t structures.
*
*   H5SC_TAG_UNPIN_WRITE_DONE_ERROR - Indicates that a chunk was unpinned
*   while handling an error during completion of a write operation. This
*   tag applies to H5SC_chunk_t structures.
*
*   H5SC_TAG_UNPIN_READ_DONE - Indicates that a chunk was unpinned after
*   successful completion of a read operation. This tag applies to
*   H5SC_chunk_t structures.
*
*   H5SC_TAG_UNPIN_READ_DONE_ERROR - Indicates that a chunk was unpinned
*   while handling an error during completion of a read operation. This
*   tag applies to H5SC_chunk_t structures.
*
*   H5SC_TAG_UNPIN_ERASE_DONE - Indicates that a chunk was unpinned after
*   successful completion of an erase operation. This tag applies to
*   H5SC_chunk_t structures.
*
*   H5SC_TAG_UNPIN_ERASE_DONE_ERROR - Indicates that a chunk was unpinned
*   while handling an error during completion of an erase operation. This
*   tag applies to H5SC_chunk_t structures.
*
*   H5SC_TAG_TEST - Catch-all tag reserved for SCC testing and
*   test-specific validation where one of the operational tags above is
*   not appropriate.
*
*****/
typedef enum H5SC_tag_t {
    H5SC_TAG_NONE = 0,
    H5SC_TAG_CREATE,
    H5SC_TAG_HT_INSERT,
    H5SC_TAG_HT_DELETE,

```

```

H5SC_TAG_LRU_INSERT,
H5SC_TAG_LRU_REMOVE,
H5SC_TAG_LRU_PROMOTE,
H5SC_TAG_LRU_TOUCH,
H5SC_TAG_FLUSH_DIRTY,
H5SC_TAG_FLUSH,
H5SC_TAG_EVICT,
H5SC_TAG_LOOKUP,
H5SC_TAG_PIN_IOINIT,
H5SC_TAG_PIN_IOINIT_CACHED,
H5SC_TAG_UNPIN_WRITE_DONE,
H5SC_TAG_UNPIN_WRITE_DONE_ERROR,
H5SC_TAG_UNPIN_READ_DONE,
H5SC_TAG_UNPIN_READ_DONE_ERROR,
H5SC_TAG_UNPIN_ERASE_DONE,
H5SC_TAG_UNPIN_ERASE_DONE_ERROR,
H5SC_TAG_TEST /* catch-all for testing */
} H5SC_tag_t;

```

2.2. H5SC_size_est_source_t – Resident-Size Estimate Source

```

/*****
 *
 * Enumeration: H5SC_size_est_source_t
 *
 * Info
 *
 * The enumeration identifies the source of a resident-size estimate
 * produced for a nonresident chunk before that chunk is admitted for
 * materialization by the SCC.
 *
 * Resident-size estimates are used by SCC batching and active-limit
 * admission logic to predict the final decoded allocation associated
 * with a selected chunk. These estimates are heuristics and are not
 * authoritative cache-accounting values. After materialization, the
 * actual layout-reported resident allocation replaces the estimate for
 * cache accounting and may be used to update dataset-local estimate
 * history.
 *
 * The source value is stored with the pending estimate in
 * H5SC_io_sel_chunk_t so that estimate accuracy can be attributed to the
 * mechanism that produced it.
 *
 * Not every source represented by this enumeration is currently active.
 * The present implementation uses dataset history when observations are
 * available and otherwise falls back to the dense logical chunk size.
 * Lookup-based and sparse-model estimates are reserved for future use.
 *
 * Values
 *
 * H5SC_SIZE_EST_NONE - Indicates that no resident-size estimate source
 * is currently associated with the selected chunk. This is the default
 * value and is restored when a pending estimate is cleared.
 *
 * H5SC_SIZE_EST_LOOKUP - Indicates that the estimate was derived from
 * layout lookup information that provides or can reliably predict the
 * complete decoded resident allocation. This source is reserved for

```

```

*         future use by the current structured-chunk implementation.
*
* H5SC_SIZE_EST_SPARSE_MODEL - Indicates that the estimate was produced
*         by a sparse-representation model derived from information about the
*         chunk's defined values or representation. This source is reserved for
*         future use by the current implementation.
*
* H5SC_SIZE_EST_DATASET_HISTORY - Indicates that the estimate was derived
*         from previously observed resident allocation sizes for chunks in the
*         same dataset. The current implementation uses the dataset-local
*         exponential moving average and decaying high-water observation to
*         produce this estimate.
*
* H5SC_SIZE_EST_DENSE_FALLBACK - Indicates that the dataset's dense
*         logical chunk size was used as the estimate because no usable
*         dataset-local resident-size history was available.
*
*****/
typedef enum H5SC_size_est_source_t {
    H5SC_SIZE_EST_NONE = 0,
    H5SC_SIZE_EST_LOOKUP,
    H5SC_SIZE_EST_SPARSE_MODEL,
    H5SC_SIZE_EST_DATASET_HISTORY,
    H5SC_SIZE_EST_DENSE_FALLBACK
} H5SC_size_est_source_t;

```

2.3. H5SC_chunk_key_t – Chunk Indexing

```

/*****
*
* Structure: H5SC_chunk_key_t
*
* Info
*
* The structure stores a unique 128-bit key for a chunk. The key is derived
* from the dataset's object header address in the file and the chunk's
* serialized logical coordinates using an LSB-first bit interleaving schema.
*
* Fields
*
* uint64_t low_half - The bottom 64-bits of the 128-bit integer key.
*
* uint64_t high_half - The top 64-bits of the 128-bit integer key.
*
* Example: Assume 64-bit dataset object header address =  $6_{10}$  or  $\dots 0110_2$ ,
*         and 64-bit chunk logical coordinate =  $5_{10}$  or  $\dots 0101_2$ .
*         LSB-first bit interleaving yields 0,1,1,0,1,1  $\rightarrow$  01101100...2
*         with remaining zeros omitted for clarity. The result of
*         the operation is stored in the two 64-bit integer fields
*         high_half = 01101100...2; low_half = 00000000...2;
*
*****/

struct H5SC_chunk_key_t {
    uint64_t high_half;
    uint64_t low_half;
} H5SC_chunk_key_t;

```

Design Notes

No major notes; this structure does what it says on the tin.

2.4. H5SC_chunk_t – Individual Cache Entry Structure

```

/*****
 *
 * Structure: H5SC_chunk_t
 *
 * Info
 *
 * The structure represents an individual chunk entry in SCC; it is an entry
 * in the chunk hash table.
 *
 * The structure stores the chunk key, pointers to in-memory and on disk chunk
 * data, chunk size, state flags, and pointers necessary to track the chunk as
 * a member of the associated dataset's chunk LRU list. A hash handle is
 * included to make the structure hashable.
 *
 * Fields
 *
 * uint64_t magic - An integer value that is set by internal functions that
 * create an instance of H5SC_chunk_t structure. Expected to be set to
 * 1396917064. This value is used to verify that an instance of a
 * H5SC_chunk_t structure was created properly.
 *
 * H5SC_chunk_key_t data_key - The unique key associated with the cached
 * chunk. It is primarily used for chunk lookups through the chunk-
 * focused hash table and operations that require internal chunk index
 * updates.
 *
 * void *chunk_obj - Pointer to the layout-specific decoded resident chunk
 * object. For the current structured-chunk implementation, this points
 * to an H5D_chunk_cache_mem_t.
 *
 * void *udata - Pointer to layout-specific callback user data associated with
 * the decoded chunk representation. The SCC treats this pointer as
 * opaque and passes it through to the applicable layout callbacks during
 * lookup, decode/encode, read/write processing, flush, erase, and
 * eviction operations. Ownership and interpretation of the pointed-to
 * object are defined by the layout implementation rather than by the SCC
 * itself. For the current structured-chunk implementation, this pointer
 * refers to an H5D_chunk_ud_t object. When a decoded chunk becomes
 * resident in the SCC, the associated udata may be retained with the
 * H5SC_chunk_t for reuse by subsequent callback operations and is
 * released as part of layout-specific resident-chunk teardown or
 * eviction.
 *
 * unsigned ndims - Number of valid dims defined in the scaled coordinates for
 * this chunk. The default value is 0, a convenient invalid value.
 *
 * hsize_t chk_log_coord - The linearized, dataset-relative chunk coordinate,
 * referred to as the chunk logical coordinate (index). This value is
 * computed from the dataset dimensions, chunk dimensions, and element
 * position. It is computed when a H5SC_chunk_t structure is added to the

```

```

*      SCC and is used as one of the components for the chunk key
*      (H5SC_chunk_key_t). This value is stored as a reference for checking
*      whether a chunk should be reindexed when a dataset contains one or
*      more extensible dimensions.
*
*      hsize_t scaled[H5S_MAX_RANK] - The scaled coordinates associated with this
*      chunk (essentially the offset divided by the chunk dimension (for each
*      rank)). This array is set during the initialization of an I/O request
*      and is used for cache flush procedures and callback operations.
*
*      haddr_t disk_addr - The on-disk address of the chunk. Set to HADDR_UNDEF
*      when the chunk is created. Necessary for cache hits to be processed
*      correctly.
*
*      size_t cached_chunk_size - Layout-reported total resident allocation
*      associated with the decoded chunk representation. The value may
*      include data storage, decoded selection metadata, and other
*      layout-owned resident allocations. It is not the encoded on-disk
*      chunk size.
*
*      Changes to this field must be routed through
*      H5SC__chunk_update_cached_size() while the chunk is tracked by SCC.
*      The default value is zero.
*
*      size_t disk_nbytes - The last known size of the on disk data buffer prior to
*      applying filters or decoding. It is obtained using the
*      H5D__struct_chunk_lookup callback. The default value is set to 0.
*
*      size_t chunk_counter - Pin count for the chunk. A nonzero value indicates
*      one or more active SCC request pins and makes the chunk ineligible for
*      reclamation. Currently used to track the pin count for a chunk. A
*      chunk is pinned to indicate it is a participant in an I/O request to
*      prevent premature removal from the SCC. Expected to be 0 when not
*      selected in an I/O request and 1 until it has been decremented during
*      a typical I/O operation. When this chunk is in an I/O request, it is
*      expected to be incremented by H5SC__io_info_init() or
*      H5SC__erase_io_info_init() during the selection process and
*      decremented by H5SC_read(), H5SC_write(), or H5SC_erase() (or within
*      H5SC__invoke_read_dset_batched() or H5SC__invoke_write_dset_batched()
*      during chunk-by-chunk debugging).
*
*      H5SC_tag_t last_op - An enum-typed tag used during debugging to identify the
*      most recent internal SCC operation performed on this chunk. Set to
*      H5SC_TAG_CREATE when an H5SC_chunk_t structure is created. See the
*      section defining H5SC_tag_t for possible values.
*
*      bool dirty_flag - A flag to indicate when an in-memory chunk buffer has been
*      modified. A value of True indicates a chunk buffer is dirty. This flag
*      is set when a chunk is created/modified/resized. Set to False by
*      default.
*
*      bool partial_IO - Indicates that the resident chunk has participated in a
*      partial read or write operation, meaning that the chunk-local
*      selection represented less than the complete logical chunk. This field
*      records SCC processing state associated with the resident
*      representation and may be used by operations that need to distinguish
*      partial-I/O state from a fully processed chunk. The field is not
*      currently used to rank chunks within the active or quiescent

```

```

*      reclamation policies. Reclamation eligibility and ordering are
*      determined separately through pin state, resident size, dirty state,
*      dataset retention policy, and LRU position.
*
*      The field is set to false by default.
*
*      struct H5SC_chunk_t *prev_ptr - The pointer to the previous node in the
*      dataset's LRU list. It is Null if this chunk is at the head of the LRU
*      list or if the LRU list has a single element.
*
*      struct H5SC_chunk_t *next_ptr - The pointer to the next node in the
*      dataset's LRU list. It is Null if this chunk is at the tail of the LRU
*      list or if the LRU list has a single element.
*
*      UT_hash_handle hval_chunk - A hash handle is required for this structure to
*      be hashable by UTHash; it is used to track this chunk in the SCC chunk
*      hash table. Set to Null by default.
*
*****/

struct H5SC_chunk_t {
    uint64_t          magic;
    H5SC_chunk_key_t  data_key;
    void              *chunk_obj;
    void              *udata;
    unsigned          ndims;
    hsize_t           chk_log_coord;
    hsize_t           scaled[H5S_MAX_RANK];
    haddr_t           disk_addr;
    size_t            cached_chunk_size;
    size_t            disk_nbytes;
    size_t            chunk_counter;
    H5SC_tag_t        last_op;
    bool              dirty_flag;
    bool              partial_IO;
    struct H5SC_chunk_t *prev_ptr;
    struct H5SC_chunk_t *next_ptr;
    UT_hash_handle     hval_chunk;
};

```

Design Notes

This structure is created for a chunk within an I/O request during the initialization step (`H5SC__io_info_init()`). Upon creation, the chunk is treated as a “placeholder chunk”; that is, a chunk that does not yet contain any cached data. As a placeholder, this chunk is protected from eviction until it has been properly initialized. This is done to ensure that a chunk present within an I/O request is not prematurely evicted before it can be utilized. Once a chunk has been updated, it is treated as a “resident chunk”. Resident chunks are eligible for eviction candidate selection. It is also worth noting that this structure contains some fields that are similar to `H5SC_io_sel_chunk_t`. For Version 1, this has largely been accepted as necessary. However, it may be worth considering merging these structures.

2.5. H5SC_io_scratch_t – Structure Used for Dynamically Sized Scratch Storage

```

/*****
*

```

```

* Structure: H5SC_io_scratch_t
*
* Info
*
*     The structure maintains reusable, dynamically sized scratch storage
*     used while processing SCC read, write, lookup, and related I/O
*     operations.
*
*     Many SCC layout callbacks accept vector arguments whose elements are
*     pointers to scalar values. Historically, these vectors and their
*     individual scalar objects were allocated separately for each I/O
*     invocation. H5SC_io_scratch_t provides reusable pointer vectors and
*     contiguous backing arrays for those values so that repeated SCC
*     operations do not require large numbers of small heap allocations.
*
*     The scratch storage is associated with an H5SC_io_info_t structure
*     and therefore with a single SCC dataset header. Capacity is grown as
*     necessary to accommodate an I/O window and is retained for reuse by
*     subsequent operations on the same dataset.
*
*     Scratch contents are transient and have no semantic meaning between
*     I/O operations. Callers must initialize the portion of each array that
*     they intend to use before invoking a layout callback. Pointers stored
*     in the vector-view fields reference the corresponding contiguous
*     backing-storage arrays and must not be independently freed.
*
*     The scratch structure does not own decoded chunk objects, layout-
*     specific udata, selected-chunk dataspace, or user buffers. Pointer
*     arrays such as udata and chunk may temporarily reference
*     such objects, but ownership remains governed by the normal SCC/layout
*     callback contracts.
*
* Lifetime
*
*     H5SC_io_scratch_t is dataset-owned through H5SC_io_info_t.
*
*     - It is allocated lazily when scratch capacity is first required.
*
*     - H5SC__io_scratch_ensure() grows the backing arrays when an operation
*       requires more entries than are currently available. Existing scratch
*       contents may be discarded when growth occurs.
*
*     - H5SC__io_info_reset() does not free or shrink scratch storage.
*       Resetting an I/O request therefore retains the allocated capacity for
*       reuse by later operations on the same dataset.
*
*     - H5SC__io_info_term() permanently releases the scratch arrays and the
*       H5SC_io_scratch_t structure when the dataset's persistent I/O state
*       is destroyed.
*
*     Scratch storage must not contain the sole owning reference to an
*     allocation when H5SC__io_info_reset() or H5SC__io_info_term() is
*     called. Any callback-generated object requiring persistent ownership
*     must first be transferred to its appropriate H5SC_chunk_t or otherwise
*     released.
*
* Fields
*

```

```

* *** General callback-vector views ***
*
*   size_t allocated - Number of entries currently available in each
*   reusable scratch vector. A value of 0 indicates that no vector
*   backing storage has been allocated.
*
*   const hsize_t **scaled - Reusable vector of pointers to scaled chunk
*   coordinates. The coordinates themselves are owned elsewhere, normally
*   by H5SC_io_sel_chunk_t or H5SC_chunk_t.
*
*   haddr_t **addr - Callback-facing vector whose entries point into
*   addr_values.
*
*   hsize_t **size - Callback-facing vector whose entries point into
*   size_values.
*
*   hsize_t **defined_values_size - Callback-facing vector whose entries
*   point into defined_values_size_values.
*
*   size_t **size_hint - Callback-facing vector whose entries point into
*   size_hint_values.
*
*   size_t **defined_values_size_hint - Callback-facing vector whose
*   entries point into defined_values_size_hint_values.
*
*   void **udata - Reusable vector for layout-specific callback udata
*   pointers. The vector does not own the objects referenced by its
*   elements.
*
* *** Contiguous backing storage for scalar callback outputs ***
*
*   haddr_t *addr_values - Contiguous scalar backing storage for addr.
*
*   hsize_t *size_values - Contiguous scalar backing storage for size.
*
*   hsize_t *defined_values_size_values - Contiguous scalar backing
*   storage for defined_values_size.
*
*   size_t *size_hint_values - Contiguous scalar backing storage for
*   size_hint.
*
*   size_t *defined_values_size_hint_values - Contiguous scalar backing
*   storage for defined_values_size_hint.
*
* *** Read/write processing state ***
*
*   size_t *miss_idx - Reusable vector containing indexes of selected
*   chunks requiring miss-path processing.
*
*   size_t *hit_idx - Reusable vector containing indexes of selected
*   chunks handled through resident-hit processing.
*
*   void **chunk - Reusable vector of chunk-object or temporary chunk-
*   buffer pointers. Ownership is determined by the processing path and
*   is not implied by storage in this vector.
*
* *** Lookup-miss batching ***
*

```

```

*      const hsize_t **lookup_scaled - Scratch vector of scaled coordinates
*      used when batching layout lookup misses.
*
*      haddr_t **lookup_addr - Callback-facing address vector used by
*      miss-only lookup batching.
*
*      hsize_t **lookup_size - Callback-facing encoded-size vector used by
*      miss-only lookup batching.
*
*      hsize_t **lookup_defined_values_size - Callback-facing defined-value
*      metadata-size vector used by miss-only lookup batching.
*
*      size_t **lookup_size_hint - Callback-facing encoded-buffer allocation
*      hint vector used by miss-only lookup batching. For the current
*      structured-chunk implementation, each nonzero value equals the
*      encoded on-disk chunk size and is used to allocate the raw buffer
*      passed to decode.
*
*      size_t **lookup_defined_values_size_hint - Callback-facing allocation
*      hint vector for the encoded defined-values metadata buffer. Values
*      are byte sizes, not defined-element counts.
*
*      void **lookup_udata - Temporary vector receiving layout-specific udata
*      from miss-only lookup batching. The vector itself is reusable; the
*      pointed-to objects are not owned by H5SC_io_scratch_t.
*
*      size_t *lookup_idx - Reusable mapping from entries in the compact
*      miss-only lookup vectors back to indexes in the active
*      H5SC_io_sel_chunk_t selection window.
*
*      size_t *order_idx - Reusable vector containing absolute indexes into
*      the owning H5SC_io_info_t::sel_chunks allocation. For read and write
*      requests, the valid prefix is populated in stable resident-first
*      order: selections whose cached chunks already contain decoded resident
*      objects appear first, followed by selections requiring lookup or
*      materialization. Relative order within each group is preserved.
*
*      The vector is owned by H5SC_io_scratch_t, allocated and grown by
*      H5SC__io_scratch_ensure(), retained across H5SC__io_info_reset(), and
*      released by H5SC__io_info_term(). Entries have no semantic validity
*      between I/O requests.
*
*****/
struct H5SC_io_scratch_t {
    size_t allocated;

    const hsize_t **scaled;
    haddr_t **addr;
    hsize_t **size;
    hsize_t **defined_values_size;
    size_t **size_hint;
    size_t **defined_values_size_hint;
    void **udata;

    haddr_t *addr_values;
    hsize_t *size_values;
    hsize_t *defined_values_size_values;
    size_t *size_hint_values;

```

```

size_t  *defined_values_size_hint_values;

size_t  *miss_idx;
size_t  *hit_idx;
void    **chunk;

const hsize_t **lookup_scaled;
haddr_t      **lookup_addr;
hsize_t      **lookup_size;
hsize_t      **lookup_defined_values_size;
size_t       **lookup_size_hint;
size_t       **lookup_defined_values_size_hint;
void         **lookup_udata;
size_t       *lookup_idx;

size_t *order_idx;
};

```

2.6. H5SC_io_info_t – Structure Used for an I/O Request on a Dataset

```

/*****
 *
 * Structure: H5SC_io_info_t
 *
 * Info
 *
 * The structure represents the state associated with a dataset I/O request
 * in SCC. It encapsulates the set of chunks participating in the operation
 * and provides bookkeeping fields to manage iteration and storage of those
 * selected chunks.
 *
 * The structure primarily serves as a container for chunk selection results
 * produced during I/O setup. It is used to track which chunks are involved in
 * an I/O operation, the starting position (used for batching), how many chunks
 * are present in the request, and how large sel_chunks currently is (in
 * bytes). These fields are subsequently consumed by I/O execution paths and
 * chunk processing routines.
 *
 * The following relations are expected for an active request:
 *
 * * 0 <= num_resident_sel_chunks <= num_sel_chunks
 * * num_sel_chunks <= sel_chunks_allocated
 * * if sel_order is NULL:
 *   sel_start + num_sel_chunks <= sel_chunks_allocated
 * * if sel_order is non-NULL:
 *   sel_order[j] < sel_chunks_allocated for every active j
 *
 * Fields
 *
 * H5SC_io_sel_chunk_t *sel_chunks - Pointer to a vector of structures
 * describing the chunks selected for the current I/O operation. Each
 * element contains per-chunk selection and metadata required to
 * execute I/O on that chunk. Default size of 128; maximum size is
 * defined by sel_chunks_allocated. The number of non-NULL elements present
 * is defined by num_sel_chunks.
 *
 * const size_t *sel_order - Non-owning view of the ordering vector used
 * to resolve the active selected-chunk window. When non-NULL,

```

```

*      sel_order[j] is the absolute index of relative selection j in the
*      stable sel_chunks allocation. When NULL, selections are resolved
*      contiguously as sel_chunks[sel_start + j].
*
*      During normal read and write initialization, this field references
*      scratch->order_idx. Invoke-layer batching may temporarily advance the
*      pointer to expose a subrange of that order vector. The pointer must
*      never be freed independently and must be restored after batched
*      processing.
*
*      H5SC_io_scratch_t *scratch - Pointer to reusable dataset-owned scratch
*      storage used by SCC I/O processing. The structure is allocated lazily
*      and its capacity is retained across H5SC__io_info_reset() calls.
*      H5SC__io_info_term() permanently releases this storage.
*
*      size_t sel_start - The starting index within sel_chunks for processing.
*      This field is typically used to support batched processing of selected
*      chunks.
*
*      size_t num_sel_chunks - The number of valid chunk entries currently
*      stored in sel_chunks. Serves as a count for the number of chunks
*      participating in the I/O request.
*
*      size_t num_resident_sel_chunks - Number of resident decoded chunk
*      selections at the beginning of the complete resident-first order
*      vector. A selection is resident when its associated H5SC_chunk_t has a
*      non-NULL chunk_obj.
*
*      The invoke-layer batching functions use this count as the boundary
*      between the resident and nonresident phases so that a batch never
*      crosses from resident processing into lookup/materialization
*      processing. This count describes the complete request order, not the
*      length of an individual active batch.
*
*      size_t sel_chunks_allocated - The total number of elements allocated for the
*      sel_chunks vector. This value defines the current maximum capacity of
*      the vector and is used to determine when reallocation is required to
*      accommodate more chunks. This value is 128 by default and increases by
*      multiples of 2 as necessary.
*
*****/
struct H5SC_io_info_t {
    H5SC_io_sel_chunk_t *sel_chunks;
    const size_t        *sel_order;
    H5SC_io_scratch_t    *scratch;
    size_t               sel_start;
    size_t               num_sel_chunks;
    size_t               num_resident_sel_chunks;
    size_t               sel_chunks_allocated;
};

```

Design Notes

2.7. H5SC_io_sel_chunk_t – Callback-Related Chunk Information for I/O Ops

```

/*****
*
* Structure: H5SC_io_sel_chunk_t

```

```

*
* Info
*
* The structure represents a single chunk participating in a dataset I/O
* request in SCC. It encapsulates all per-chunk state required to map
* between dataset coordinates, chunk indexing, memory/file dataspace,
* and any associated cached chunk entry used during I/O execution.
*
* The structure serves as the fundamental unit of work for chunked I/O
* operations. It binds together spatial information (coordinates and
* scaled indices), cache state, and dataspace selections needed to
* correctly perform partial or full reads/writes on an individual chunk.
* Lookup-related fields are included to support lookup-based behavior.
*
* Fields
*
* H5D_dset_io_info_t *dset_info - Pointer to the dataset I/O context for
* the dataset this chunk belongs to. Provides access to layout,
* filter pipeline, and other dataset-level information required
* for I/O execution.
*
* hsize_t coords[H5S_MAX_RANK] - The absolute coordinates of the chunk
* within the dataset's logical space. These correspond to the
* chunk's offset in each dimension and are given from the upper left
* hand corner.
*
* hsize_t scaled[H5S_MAX_RANK] - The scaled chunk coordinates, computed
* by dividing coords by the chunk dimensions for each rank. These
* values are used for chunk indexing, hashing, and cache lookup
* operations.
*
* hsize_t chk_log_coord - The linearized, dataset-relative chunk coordinate,
* referred to as the chunk logical coordinate (index). This value is
* computed from the dataset dimensions, chunk dimensions, and element
* position. It is computed when a H5SC_chunk_t structure is added to the
* SCC and is used as one of the components for the chunk key
* (H5SC_chunk_key_t). This value is stored as a reference for checking
* whether a chunk should be reindexed when a dataset contains one or
* more extensible dimensions.
*
* H5SC_chunk_t *cached_chunk - Pointer to the corresponding SCC cache
* entry for this chunk. This is populated when the chunk is found
* or inserted into the cache and is NULL if no cache entry has
* been associated yet.
*
* bool pin_held - Indicates that this selection entry owns one increment
* of cached_chunk->chunk_counter. The flag is set immediately after the
* selection initialization path increments the chunk counter and is
* cleared immediately after the corresponding read, write, erase, or
* error-cleanup path decrements it.
*
* This field tracks per-selection ownership and must be used when
* releasing pins; chunk_counter > 0 alone does not prove that a
* particular selection owns a pin. A selection must not decrement the
* chunk counter when pin_held is false.
*
* bool lookup_valid - Indicates whether the lookup-related fields
* contain valid information for this chunk.

```

```

*
* haddr_t lookup_addr - The on-disk address obtained from lookup
*                      operations.
*
* size_t lookup_disk_nbytes - Last known encoded on-disk size of the
*                             structured chunk, after application of any applicable filter
*                             pipelines. The value is obtained from layout lookup or updated after
*                             insertion. It does not describe the decoded resident allocation. The
*                             default value is zero.
*
* hsize_t lookup_defined_values_size - Encoded byte size occupied by
*                                     defined-values selection metadata returned by lookup. This is a byte
*                                     count, not a count of defined elements.
*
* size_t lookup_size_hint - Layout-provided allocation hint for the encoded
*                           chunk buffer used by the raw read/decode path. For the current
*                           structured-chunk layout implementation, this value is the encoded
*                           on-disk chunk size returned by lookup.
*
*                               This value is a byte size, not a defined-value count, and does not
*                               describe the complete decoded resident allocation. It must not be used
*                               as H5SC_SIZE_EST_LOOKUP unless a future layout callback provides a
*                               separately documented complete resident-allocation estimate.
*
* size_t lookup_defined_values_size_hint - Layout-provided allocation hint
*                                         for reading the encoded defined-values selection metadata. For the
*                                         current structured-chunk implementation, this value equals
*                                         lookup_defined_values_size.
*
*                               It is an encoded byte size, not a count of defined elements and not a
*                               complete decoded resident-allocation estimate.
*
* void *lookup_udata - Pointer to user data structure used during
*                     lookup callbacks.
*
* size_t estimate_resident_size - Predicted final resident allocation size
*                               for this selected chunk. The invoke-layer batching code sets this
*                               before admitting a nonresident chunk into a batch. The value remains
*                               valid only while estimate_pending is true and is reset when the
*                               estimate is recorded or the request state is reset.
*
* H5SC_size_est_source_t estimate_source - Identifies the source used to
*                                         produce estimate_resident_size. In the current implementation,
*                                         estimates are produced from dataset-local resident-size history when
*                                         available and otherwise from the dense logical chunk size.
*                                         H5SC_SIZE_EST_LOOKUP and H5SC_SIZE_EST_SPARSE_MODEL are reserved for
*                                         future estimation mechanisms and are not currently selected by
*                                         H5SC__estimate_resident_size(). Set to H5SC_SIZE_EST_NONE when no
*                                         estimate is pending.
*
* bool estimate_pending - Indicates that estimate_resident_size and
*                        estimate_source describe a materialization estimate that has not yet
*                        been compared with an actual resident allocation. A successful read
*                        materialization or write gather records the estimate and clears this
*                        flag. Failed or abandoned requests clear it during request reset
*                        without recording an observation.
*
* H5S_t *file_space - Dataspace describing the selection within the

```

```

*      chunk in file coordinates. The extent matches the chunk
*      dimensions, and the selection represents the portion of the
*      chunk involved in the I/O operation.
*
*      H5S_t *mem_space - Dataspace describing the selection within the
*      user's memory buffer corresponding to this chunk. The extent
*      matches the overall memory dataspace, with the selection
*      restricted to the elements mapped to this chunk.
*
*      bool file_space_shared - Ownership flag for file_space. A value of false
*      indicates that the dataspace is a temporary per-chunk selection owned
*      by the SCC and must be closed by SCC request cleanup. A value of true
*      indicates that file_space is a borrowed reference to a dataspace owned
*      elsewhere; SCC may use the dataspace for the current selection record
*      but must not close it. The current read/write initialization path
*      constructs an SCC-owned file_space for each selected chunk, so this
*      flag normally remains false. The flag exists to distinguish owned
*      temporary dataspace references from borrowed dataspace references if such reuse
*      is employed by a processing path.
*
*      bool mem_space_shared - Ownership flag for mem_space. A value of false
*      indicates that the dataspace is a temporary per-chunk memory selection
*      created and owned by the SCC and must be closed by SCC request
*      cleanup. A value of true indicates that mem_space is a borrowed
*      reference to a dataspace owned elsewhere and must not be closed by the
*      SCC. In the current implementation, this occurs when an I/O request
*      selects exactly one chunk: the selected-chunk record directly
*      references dset_info->mem_space rather than creating a separate per-
*      chunk memory dataspace.
*
*      H5_flexible_const_ptr_t buf - Pointer to the user-provided memory
*      buffer used for I/O. This buffer is used as the source or
*      destination for data transferred between memory and the chunk.
*
*****/
struct H5SC_io_sel_chunk_t {
    H5D_dset_io_info_t    *dset_info;
    hsize_t               coords[H5S_MAX_RANK];
    hsize_t               scaled[H5S_MAX_RANK];
    hsize_t               chk_log_coord;
    H5SC_chunk_t          *cached_chunk;
    bool                  lookup_valid;
    bool                  pin_held;
    haddr_t               lookup_addr;
    hsize_t               lookup_disk_nbytes;
    hsize_t               lookup_defined_values_size;
    size_t                lookup_size_hint;
    size_t                lookup_defined_values_size_hint;
    void                  *lookup_udata;
    size_t                estimate_resident_size;
    H5SC_size_est_source_t estimate_source;
    bool                  estimate_pending;
    H5S_t                 *file_space;
    H5S_t                 *mem_space;
    bool                  file_space_shared;
    bool                  mem_space_shared;
    H5_flexible_const_ptr_t buf;
};

```

Design Notes

This structure is a hold-over from the Version 0 functionality. It will likely be combined into H5SC_chunk_t, however this has been reserved as an optimization for the time being.

2.8. H5SC_dset_header_t – Dataset-Based Chunk Tracking

```

/*****
 *
 * Structure: H5SC_dset_header_t
 *
 * Info
 *
 * The header structure for a dataset with data held in the shared chunk cache.
 * It tracks the dataset identity (represented by the dataset object header
 * address), the minimum amount of chunk data that should be maintained in this
 * dataset (in bytes), accounts for the total size of cached chunks. The
 * structure contains pointers to the head and tail of the LRU list for the
 * cached chunks in this dataset, pointers to the next and previous datasets in
 * the dataset LRU list, and pointers which are necessary for eviction
 * management.
 *
 * Fields
 *
 * uint64_t magic - Value set when a H5SC_dset_header_t structure is created.
 * This value is used for debugging to ensure that the structure is
 * created/destroyed consistently.
 *
 * haddr_t dset_addr - The 64-bit address of the object header for this
 * dataset. This field is used as the key for the dataset hash table.
 * It is set to 0 by default.
 *
 * min_dset_size - Preferred minimum resident byte count retained for this
 * dataset while trimming SCC to its quiescent limit. It is a steady-
 * state retention target, not a hard reservation. Active-pressure
 * eviction may reduce the dataset below this value when necessary to
 * satisfy SCC_active_limit.
 *
 * size_t chunk_lru_len - The number of chunks present within the SCC. This
 * value is used as the running count needed for DLL operations. Set to 0
 * by default.
 *
 * size_t curr_dset_size - The current amount of chunk data, in bytes
 * associated with this dataset within the cache (i.e., is a sum of
 * current chunk sizes). This quantity is updated when a chunk is added,
 * removed, or resized. This field contributes to total cache
 * utilization tracked in H5SC_t. It is set to 0 by default.
 *
 * size_t resident_estimate_high - Decaying high-water observation of final
 * resident chunk allocation sizes for this dataset. Used together with
 * resident_estimate_ema to produce the dataset-history estimate after
 * one or more successful resident-size observations have been recorded.
 * Updated only after successful final resident-size accounting.
 *
 * size_t resident_estimate_ema - Exponential moving average of observed final
 * resident chunk allocation sizes for this dataset. Used together with

```

```

*         resident_estimate_high to form the current dataset-history estimate
*         when resident-size observations are available.
*
*         uint64_t resident_estimate_samples - Number of resident allocation
*         observations incorporated into the dataset-local adaptive estimate.
*         Saturates at UINT64_MAX.
*
*         size_t reclaimable_clean_bytes - Number of resident bytes currently held
*         by unpinned, nonzero, clean chunks in this dataset that may be evicted
*         under active-pressure policy. This value does not apply
*         min_dset_size; that field is only a quiescent-retention target.
*
*         size_t reclaimable_dirty_bytes - Number of resident bytes currently held
*         by unpinned, nonzero, dirty chunks in this dataset that may be
*         reclaimed by flush-and-evict under active-pressure policy. This value
*         does not apply min_dset_size.
*
*         H5SC_tag_t last_op - An enum-typed tag used during debugging to identify the
*         most recent internal SCC operation performed on this dataset header.
*         Tags representing operations common to chunks and dataset headers are
*         interpreted in the context of the structure containing the tag. See
*         the section defining H5SC_tag_t for possible values.
*
*         H5SC_io_info_t *io_info - Structure used for an I/O request that includes
*         chunks from this dataset. This structure is populated by
*         H5SC__io_info_init() and is used by H5SC_read() and H5SC_write().
*
*         bool resize_in_progress - A flag used to indicate that a resize operation is
*         being conducted on this dataset. A value of True is set if dataset is
*         being resized. While true, any internal SCC
*         procedures and any I/O requests for this dataset should be interrupted
*         until the resize operation is completed. It is set to false by
*         default.
*
*         H5D_t *dset - Pointer to the dataset; included for debugging convenience.
*
*         H5SC_chunk_t *lru_head_ptr - The pointer to the head of this dataset's chunk
*         LRU list (indicating it is the most recently used chunk). Set to
*         Null by default or when the chunk LRU list is empty.
*
*         H5SC_chunk_t *lru_tail_ptr - The pointer to the tail of this dataset's chunk
*         LRU list (indicating it is the least recently used chunk). Set to Null
*         by default or when the chunk LRU list is empty.
*
*         H5SC_dset_header_t *next_dset_ptr - The pointer to the next dataset in the
*         dataset LRU list. Set to Null by default, when this header is at the
*         tail of the dataset LRU list, or when the list has a single element.
*
*         H5SC_dset_header_t *prev_dset_ptr - The pointer to the previous dataset in
*         the dataset LRU list. Set to Null by default, when this header is at
*         the head of the dataset LRU list, or when the list has a single
*         element.
*
*         UT_hash_handle dset_hval - A hash handle is required for this structure to
*         be hashable by UTHash; it is used to track this dataset in the SCC
*         dataset hash table. Set to Null by default.
*
*****/

```

```

struct H5SC_dset_header_t {
    uint64_t      magic;
    haddr_t      dset_addr;
    size_t        min_dset_size;
    size_t        chunk_lru_len;
    size_t        curr_dset_size;
    size_t        resident_estimate_high;
    size_t        resident_estimate_ema;
    uint64_t      resident_estimate_samples;
    size_t        reclaimable_clean_bytes;
    size_t        reclaimable_dirty_bytes;
    H5SC_tag_t    last_op;
    H5SC_io_info_t *io_info;
    bool          resize_in_progress;
    struct H5D_t  *dset;
    struct H5SC_chunk_t *lru_head_ptr;
    struct H5SC_chunk_t *lru_tail_ptr;
    struct H5SC_dset_header_t *next_dset_ptr;
    struct H5SC_dset_header_t *prev_dset_ptr;
    UT_hash_handle dset_hval;
};

```

Design Notes

This structure is created when a dataset is either created (through `H5Dcreate()`, with `H5D__create()` calling `H5SC_dset_create_header()`) or when a dataset is opened (`H5D__open_oid()` will call `H5SC_dset_create_header()`). This structure is also added to the dataset hash table and the dataset LRU list. There is no other valid way of creating an instance of `H5SC_dset_header_t`. When a dataset is closed (with `H5Dclose()`, leading to `H5D_close()` being called), the structure is removed from the dataset LRU list, removed from the dataset hash table, and the header is destroyed (with `H5SC_dset_destroy_header()`).

2.9. H5SC_stats_t – Per-File Global Cache Statistics

```

/*****
 *
 * Structure: H5SC_stats_t
 *
 * Purpose
 *     Maintains debugging and test statistics for a single SCC instance.
 *     These counters are used to observe cache behavior, particularly the
 *     rate at which chunk lookups resolve to resident non-placeholder
 *     chunks already present in the SCC.
 *
 * Fields
 *
 *     uint64_t scc_lookups - The total number of SCC chunk lookup operations
 *                           recorded for this cache instance.
 *
 *     uint64_t scc_hits - The number of SCC chunk lookups that resolved to a
 *                        resident non-placeholder chunk already present within the cache.
 *
 *     uint64_t scc_misses - The number of SCC chunk lookups that did not resolve

```

```

*         to a resident non-placeholder chunk already present within the cache.
*
* uint64_t scc_inserts - The number of chunk insert operations recorded for
*         this cache instance.
*
* uint64_t scc_removes - The number of chunk remove operations recorded for
*         this cache instance.
*
* uint64_t scc_chunk_flush_count - The number of chunk flush operations
*         recorded for this cache instance.
*
* uint64_t scc_dataset_flush_count - The number of dataset flush operations
*         recorded for this cache instance.
*
* uint64_t scc_evictions - The number of chunk or dataset eviction operations
*         recorded for this cache instance.
*
* uint64_t scc_reads_batch_calls_count - The number of individual read batches
*         recorded for this cache instance.
*
* uint64_t scc_reads_last_batch_len - The number of chunks processed in the
*         last processed read batch for this cache instance.
*
* uint64_t scc_reads_max_batch_len - The largest number of chunks processed in
*         a single read batch for this cache instance.
*
* uint64_t scc_writes_batch_calls_count - The number of individual write
*         batches recorded for this cache instance.
*
* uint64_t scc_writes_last_batch_len - The number of chunks processed in the
*         last processed write batch for this cache instance.
*
* uint64_t scc_writes_max_batch_len - The largest number of chunks processed
*         in a single write batch for this cache instance.
*
* uint64_t scc_oversized_admission_count - Number of indivisible chunks
*         admitted through the oversized single-chunk exception.
*
* size_t scc_oversized_admission_max_bytes - Largest incremental resident-
*         size estimate admitted through the exception.
*
* size_t scc_oversized_admission_max_excess - Largest amount by which an
*         oversized estimate exceeded SCC_active_limit. This may be zero when
*         the exception was triggered by reduced effective budget rather than by
*         an estimate larger than the configured limit.
*
* uint64_t size_estimate_count - Number of pending resident-size estimates
*         compared with a successfully materialized final resident allocation.
*
* uint64_t size_estimate_under_count - Number of comparisons for which the
*         estimate was less than the final resident allocation.
*
* uint64_t size_estimate_over_count - Number of comparisons for which the
*         estimate was greater than the final resident allocation. Exact
*         estimates are included only in size_estimate_count.
*
* size_t size_estimated_total - Saturating sum of all compared resident-size
*         estimates.

```

```

*
*   size_t size_actual_total - Saturating sum of the corresponding final
*       resident allocations.
*
*   size_t size_overestimated_bytes - Saturating sum of estimate - actual for
*       overestimates.
*
*   size_t size_underestimated_bytes - Saturating sum of actual - estimate for
*       underestimates.
*
*   size_t size_estimate_max_over - Largest observed estimate - actual
*       difference.
*
*   size_t size_estimate_max_under - Largest observed actual - estimate
*       difference.
*
*   uint64_t size_estimate_*_count - Number of recorded comparisons attributed
*       to each estimate source. The sum of the source counts should equal
*       size_estimate_count.
*
*   uint64_t write_growth_count - Number of materialized writes for which
*       final resident size growth was measured.
*
*   size_t write_growth_total - Saturating sum of measured write-growth bytes.
*
*   size_t write_growth_max - Largest measured write-growth value.
*
*****/

struct H5SC_stats_t {
    uint64_t scc_lookups;
    uint64_t scc_hits;
    uint64_t scc_misses;

    uint64_t scc_inserts;
    uint64_t scc_removes;

    uint64_t scc_chunk_flush_count;
    uint64_t scc_dataset_flush_count;
    uint64_t scc_evictions;

    uint64_t scc_reads_batch_calls_count;
    uint64_t scc_reads_last_batch_len;
    uint64_t scc_reads_max_batch_len;

    uint64_t scc_writes_batch_calls_count;
    uint64_t scc_writes_last_batch_len;
    uint64_t scc_writes_max_batch_len;

    uint64_t scc_oversized_admission_count;
    size_t   scc_oversized_admission_max_bytes;
    size_t   scc_oversized_admission_max_excess;

#ifdef H5SC_COLLECT_ESTIMATE_STATS && (H5SC_COLLECT_ESTIMATE_STATS + 0)
    /* General Size Estimate Stats*/
    uint64_t size_estimate_count;
    uint64_t size_estimate_under_count;

```

```

uint64_t size_estimate_over_count;

size_t size_estimated_total;
size_t size_actual_total;
size_t size_overestimated_bytes;
size_t size_underestimated_bytes;

size_t size_estimate_max_over;
size_t size_estimate_max_under;

/* Specific Estimate Source Stats */
uint64_t size_estimate_lookup_count;
uint64_t size_estimate_sparse_model_count;
uint64_t size_estimate_dense_fallback_count;
uint64_t size_estimate_dataset_history_count;

/* Write Specific Estimate Stats*/
uint64_t write_growth_count;
size_t   write_growth_total;
size_t   write_growth_max;
#endif
};

```

Design Notes

These fields are primarily intended for development, testing, and performance characterization and should not be interpreted as part of the public SCC interface.

2.10. H5SC__cache_config_t – Shared Chunk Cache Configuration

```

/*****
 *
 * struct H5SC__cache_config_t
 *
 * The H5SC__cache_config_t structure is used to aggregate shared chunk cache
 * configuration data. The fields in the structure are
 * discussed individually below.
 *
 * version:      int field containing the version number of this version
 *                of the H5SC__cache_config_t structure. Any instance of
 *                H5SC__scc_config_t passed to the cache must have a known
 *                version number, or an error will be flagged.
 *
 * max_q_size:   size_t field containing the maximum number of bytes that
 *                the shared chunk cache may use while quiescent -- that is
 *                when not actively engaged in serving an I/O request
 *
 * max_a_size:   size_t field containing the normal maximum number of resident
 *                bytes that the shared chunk cache may use while serving an I/O
 *                request. If one indivisible chunk cannot be processed within this
 *                limit, the SCC may temporarily exceed max_a_size for the duration
 *                of that single-chunk operation. Before admitting the chunk, the
 *                SCC reclaims eligible resident data under the active-pressure
 *                policy and returns to normal configured-limit enforcement after
 *                the chunk is processed and unpinned. Applications using large
 *                logical chunks should configure an active limit sufficient for the
 *                expected working set.
 *
 *****/

```

```

*****/

typedef struct H5SC__cache_config_t {

    int version;

    size_t max_q_size;

    size_t max_a_size;

} H5SC__cache_config_t;

```

Design Notes

This structure may be expanded in future updates to accommodate additional user-specified configuration data for the Shared Chunk Cache.

2.11. H5SC_t – Per-File Global Cache Management

```

/*****
 *
 * Structure: H5SC_t
 *
 * Info
 *
 * The primary structure used for shared chunk cache management. Provides
 * fields for maintaining access to the head and tail of the dataset LRU
 * list, tracking the active and quiescent cache memory usage, and the
 * pointers necessary to access the hash tables used for chunk and
 * dataset lookups.
 *
 * Fields
 *
 * uint64_t SCC_magic - Value set when a H5SC_t structure is created.
 * This value is used for debugging to ensure that the structure is
 * created/destroyed consistently.
 *
 * size_t SCC_quiescent_size - The amount of quiescent data, in bytes, present
 * within the cache when there are no I/O operation being performed on
 * cached data. This field is computed using the sum of the
 * curr_dset_size field from each dataset currently in the SCC. This
 * field is updated whenever a chunk is added, removed, or resized while
 * present within the SCC. A value of 0 indicates the cache is empty. A
 * value of 0 is set by default when this structure is created.
 *
 * size_t SCC_quiescent_limit - The amount of quiescent data, in bytes, that
 * should be retained within the cache when there is no I/O operation
 * being performed on cached data. This field is defined to be 1 GB by
 * default and may be set using a FAPL configuration (using the
 * max_q_size field in a H5SC__cache_config_t structure.)
 *
 * size_t SCC_active_size - Reserved accounting field for transient SCC
 * allocations that are not included in SCC_quiescent_size. Current chunk
 * resident-size accounting is maintained through SCC_quiescent_size, and

```

```

*      admission uses SCC_active_limit together with cached active-policy
*      reclaimability. This field is not currently part of that admission
*      calculation and remains reserved for future transient-memory
*      accounting.
*
*      size_t SCC_active_limit - Normal operational ceiling used when admitting
*      additional resident chunk bytes during an I/O request. If current
*      resident bytes plus the requested incremental allocation exceed this
*      limit, H5SC__ensure_space() performs active-pressure reclamation.
*      Active-pressure reclamation may reduce a dataset below min_dset_size.
*
*      If one indivisible chunk cannot fit within the effective active
*      budget, the SCC may temporarily exceed this limit while processing
*      that chunk. The configured limit is not modified, and ordinary active-
*      limit enforcement is restored immediately after the oversized chunk is
*      uninned. The value is configured by max_a_size.
*
*      SCC_active_limit is the normal operational ceiling rather than an
*      absolute prohibition against processing an individual chunk larger
*      than the available active budget. When a single indivisible chunk
*      cannot otherwise be admitted, SCC may temporarily exceed this limit
*      through the oversized single-chunk admission path. The configured
*      value is not modified by this exception; normal active-limit
*      enforcement resumes after the oversized chunk is processed and
*      uninned.
*
*      size_t dset_lru_len - The number of datasets present within the SCC. This
*      value is used as the running count needed for DLL debugging and
*      testing.
*
*      size_t reclaimable_clean_bytes - Cache-wide sum of dataset-local clean
*      bytes reclaimable under active-pressure policy. The value includes all
*      uninned, nonzero, clean resident chunks and does not enforce
*      dataset-local min_dset_size.
*
*      size_t reclaimable_dirty_bytes - Cache-wide sum of dataset-local dirty
*      bytes reclaimable through flush-and-evict under active-pressure
*      policy. The value includes all uninned, nonzero, dirty resident
*      chunks and does not enforce dataset-local min_dset_size.
*
*      H5SC_stats_t stats - Per-cache statistics used for SCC debugging and
*      testing. Tracks cache lookups, hits, misses, inserts, removes,
*      flushes, and evictions. These values are reset when the SCC is
*      created, destroyed, or reinitialized.
*
*      H5SC_dset_header_t *dset_lru_head_ptr - The pointer to the head of the
*      dataset LRU list (indicating it is the most recently used dataset).
*      Set to Null by default or when the dataset LRU list is empty.
*
*      H5SC_dset_header_t *dset_lru_tail_ptr - The pointer to the tail of the
*      dataset LRU list (indicating it is the least recently used dataset).
*      Set to be Null by default or when the dataset LRU list is empty.
*
*      H5SC_chunk_t *chunk_hash_table_head_ptr - The pointer to the head of the SCC
*      chunk hash table. Set to Null when the hash table is empty or by
*      default. This pointer will only change when the first element of this
*      hash table is added or removed.
*

```

```

*   H5SC_dset_header_t *dset_hash_table_head_ptr - The pointer to the head of
*   the SCC dataset hash table. Set to Null when this hash table is empty
*   or by default. This pointer will only change when the first element of
*   this hash table is added or removed.
*
* #if H5SC_DO_SANITY_CHECKS
*   bool test_fail_next_chunk_flush - Test-only one-shot injection flag.
*   When true, the next attempt to flush a dirty chunk fails before
*   encoding or storage metadata changes occur. The flush path resets the
*   field to false when consuming the injected failure.
* #endif
*
*****/

struct H5SC_t {
    uint64_t      SCC_magic;
    size_t        SCC_quiescent_size;
    size_t        SCC_quiescent_limit;
    size_t        SCC_active_size;
    size_t        SCC_active_limit;
    size_t        dset_lru_len;
    size_t        reclaimable_clean_bytes;
    size_t        reclaimable_dirty_bytes;
    H5SC_stats_t  stats;
    H5SC_dset_header_t *dset_lru_head_ptr;
    H5SC_dset_header_t *dset_lru_tail_ptr;
    H5SC_chunk_t   *chunk_hash_table_head_ptr;
    H5SC_dset_header_t *dset_hash_table_head_ptr;

#if H5SC_DO_SANITY_CHECKS
    /*
     * Test-only one-shot failure injection. When true, the next dirty-chunk
     * flush fails before encoding begins and resets this field to false.
     */
    bool test_fail_next_chunk_flush;
#endif
};

```

Design Notes

This structure is created when the file object is created (H5Fcreate()/H5Fopen()) calls H5F__new(), which calls H5SC_create(). At the time of creation, a configuration structure may be passed to H5Fcreate() to set SCC_quiescent_limit and SCC_active_limit. When the file is closed (typically with H5Fclose()), this structure is destroyed with H5SC_destroy(). The H5SC_t associated with a file is stored in the shared file info.

3 Theory of Operations

This section provides a conceptual overview of the primary operations involving the Shared Chunk Cache, including file and dataset initialization, read/write processing, erase and extent-change operations, flushing, filtering, and cache teardown.

3.1. File Open

At the time an HDF5 file is opened, `H5SC_create()` will be called to perform the following actions:

- Allocate and initialize the file specific instance of `H5SC_t` with default configuration.
 - This is done using the `H5SC_create` function, which is called from `H5F__new()` as a part of the `H5Fcreate()` call stack.
- Examine the FAPL for shared chunk cache configuration. If found, overwrite configuration fields in the instance of `H5SC_t` as specified.
- Read the SCC configuration associated with the FAPL and initialize the quiescent and active cache limits from the resulting `H5SC__cache_config_t`. When no user override is supplied, the current default SCC configuration is used.

3.2. Dataset Open

The shared chunk cache must be informed whenever a relevant¹ dataset is either opened or created. On receipt of this notification, the shared chunk cache must:

- Allocate and initialize an instance of `H5SC_dset_header_t`.
 - This is done within `H5D__create()`, which is called as a part of the public `H5Dcreate2()` call stack. The DAPL is available within `H5D__create()`.
- Scan the dataset access property list (DAPL) for non-default dataset configuration data, and if found, copy it into the instance of `H5SC_dset_header_t`.
- Insert the new instance of `H5SC_dset_header_t` into the shared chunk cache dataset hash table and dataset LRU list.

3.3. Dataset Flush

When directed to flush data associated with a dataset, the shared chunk cache must:

- Find the target instance of `H5SC_dset_header_t` in the dataset hash table in the file-specific instance of `H5SC_t`.
- Scan the chunk LRU list associated with the target instance of `H5SC_dset_header_t`, beginning at the least-recently-used end of the list.
- For each instance of `H5SC_chunk_t` on this list:
 - If the chunk is dirty, flush the resident chunk representation to persistent storage.
 - As part of the flush operation, perform any required chunk lookup, encode the resident chunk representation, update or allocate the corresponding on-disk chunk-index entry, and write the encoded data to file.

¹ Datasets are supported if their I/O is managed by the shared chunk cache. Initially this means only sparse datasets. However, as the shared chunk cache is integrated into the HDF5 library, all dataset types will become relevant.

- After a successful write, update the chunk's on-disk address and size information and mark the instance of `H5SC_chunk_t` as clean.
- If the chunk is already clean, no write is required.
- If the entry is an empty shell chunk with no resident object and no defined on-disk address, no flush operation is required.
- If `evict_after_flush` is `false`:
 - Retain all chunk entries in the SCC after dirty chunks have been flushed.
 - Preserve the existing dataset-local chunk LRU ordering and chunk hash-table membership.
 - Retain successfully flushed chunks as clean resident entries for subsequent cache reuse.
- If `evict_after_flush` is `true`:
 - Flush each dirty chunk before attempting to evict it.
 - Evict all chunk entries associated with the dataset after they are clean, including entries that were already clean and empty shell entries.
 - Remove evicted chunks from the dataset-local chunk LRU and the SCC chunk hash table and release any resident layout-specific chunk state.
 - On successful completion, leave the dataset's chunk LRU empty.
- If flushing a dirty chunk fails, do not evict that dirty chunk; return an error and preserve its dirty state so that unwritten data is not silently discarded.

3.4. Dataset Erase

When performing an erase operation through the SCC, `H5Derase()` is used to remove or invalidate a selected region of a dataset without modifying the remaining in-bounds elements of the affected chunks. When a dataset uses structured chunk storage, the following actions will be taken with respect to the dataset's chunks:

- Convert the erase request into a list of erase operations on individual chunks and identify any chunks currently resident in the shared chunk cache.
- For each affected chunk:
 - Determine whether the erase selection fully covers the chunk or only partially overlaps it.
 - If the erase selection fully covers the chunk and the resulting chunk state no longer contains defined values, remove the chunk from the SCC chunk hash table and dataset LRU, delete the chunk from storage, and evict it from cache.
 - If the erase selection only partially overlaps the chunk, preserve all non-erased elements while removing the selected region according to the dataset erase and fill-value semantics.
- If an affected chunk is already resident in cache:

- Apply the erase operation directly to the cached chunk image.
- Update the chunk's dirty state and move the corresponding instance of `H5SC_chunk_t` to the head of its per-dataset LRU.
- If an affected chunk is not resident in cache but exists on disk:
 - Read the chunk's on-disk image.
 - Decode the chunk and apply any configured filters to reconstruct the in-memory representation.
 - Apply the erase operation to the selected region.
 - Re-encode and rewrite the chunk if the chunk still contains defined values, otherwise delete the chunk from storage.
- If an erase request targets a chunk or region that has never been materialized:
 - Avoid unnecessary chunk creation or materialization when the erase operation would not change the logical dataset contents.
 - Preserve sparse-storage behavior by skipping undefined regions whenever possible.
- Throughout erase processing, maintain SCC cache invariants by updating per-dataset LRUs, chunk hash-table membership, resident chunk accounting, dirty state tracking, and cached chunk metadata to reflect the resulting chunk state.

When the layout's `erase_values` callback reports that all defined values have been removed from a chunk by setting `delete_chunk` to true, SCC immediately removes that chunk rather than deferring deletion until a later flush. Any resident chunk representation is released, the chunk is removed from SCC tracking, and its persistent storage/index entry is deleted through the layout's `delete_chunk` callback. Partial erases that leave defined values in the chunk update the resident representation and leave the chunk dirty for subsequent persistence.

3.5. Applying Filters Through the SCC

When using filters with datasets routed through the SCC, filter processing is integrated directly into the structured chunk I/O path. Filters are configured on the dataset creation property list associated with a structured chunk dataset and participate transparently in normal SCC chunk lifecycle operations, including chunk materialization, cache residency management, eviction, erase processing, extent transitions, and reopen persistence.

When operating on a filtered structured chunk dataset, the following actions are performed with respect to the dataset's chunks:

- Configure structured chunk storage through `H5Pset_layout()` and `H5Pset_struct_chunk()`.
- Install one or more filters through `H5Pset_filter2()` for the desired structured chunk section type.
- During chunk materialization or load:

- Locate the target chunk through the SCC lookup and structured chunk indexing path.
- Read the chunk's encoded on-disk image if the chunk is not already resident in cache.
- Decode the chunk image and reconstruct the in-memory chunk representation through the configured filter pipeline.
- Insert or promote the corresponding instance of `H5SC_chunk_t` within the SCC residency structures.
- During chunk flush or writeback:
 - Construct the chunk's serialized on-disk representation.
 - Apply the configured filter pipeline to the chunk buffer.
 - Write the encoded chunk image to storage.
 - Update SCC accounting and chunk residency state to reflect the resulting chunk state.
- If a filtered chunk is already resident in cache:
 - Operate directly on the decoded in-memory chunk representation.
 - Avoid unnecessary rematerialization or decode work while the chunk remains resident.
- If a filtered chunk must be evicted:
 - Flush and encode dirty chunk state as necessary.
 - Remove the corresponding chunk from the active SCC residency structures.
 - Preserve consistency between chunk hash membership, dataset LRUs, and SCC accounting state.
- Throughout filtered SCC operation:
 - Filtered datasets participate in the same batching, cache-accounting, and eviction workflows as unfiltered datasets.
 - Filtered chunks participate in erase processing, extent shrink/regrow handling, sparse chunk materialization, reopen persistence, and multidimensional chunk indexing workflows.
 - Temporary transient memory may be required during decode and re-encode operations while filtered chunks are resident in cache.
- Multiple filters may be applied to the same structured chunk dataset, and different structured chunk section types may participate in different filter pipelines depending on dataset configuration.

The current structured-chunk implementation disables HDF5 selection I/O when a structured-chunk filter pipeline is present. During structured-chunk I/O initialization, the presence of one or more configured filter sections sets selection I/O mode to off and records

H5D_SEL_IO_DATASET_FILTER as the cause. Filtered structured-chunk operations remain supported through the SCC's resident/materialization paths; disabling selection I/O does not imply that filtered SCC I/O is unsupported.

3.6. Dataset Change Extent

- When performing a dataset extent change operation through the SCC, the upper layers of the library will first update the dataset extent information, then invoke `H5SC_set_extent_notify()`. When a dataset uses structured chunk storage, the following actions will be taken with respect to the dataset's chunks:
 - Locate the target instance of `H5SC_dset_header_t` within the file-specific instance of `H5SC_t` and iterate over all resident chunks in the dataset LRU.
 - For each resident chunk:
 - Determine whether the chunk is now fully outside the updated dataset extent, partially outside the extent, or still fully valid.
 - If the chunk is fully outside the extent, remove it from the SCC chunk hash table and dataset LRU, delete the chunk from storage, and evict it from cache.
 - If the chunk partially overlaps the new extent boundary, erase or fill the newly invalid region according to the dataset fill-value and erase semantics while preserving all still-valid in-bounds elements.
- After resident pruning is complete, recompute the logical chunk index for all remaining resident chunks using the updated dataset dimensions. Remove each chunk from the SCC chunk hash table, update its logical chunk coordinate and associated hash-table key, then reinsert it into the hash table under the new index.
- Perform a second pruning pass over chunks that may exist only in storage and are not currently resident in cache:
 - Identify chunks intersecting the region removed from the dataset extent.
 - If a chunk lies completely outside the new extent, delete it directly from storage.
- If the extent grows, SCC does not materialize newly introduced chunks. If the resulting chunk-grid dimensions change, surviving resident chunks are rekeyed using the updated logical chunk indexing.
- Throughout extent processing, maintain SCC cache invariants by updating per-dataset LRUs, chunk hash-table membership, resident chunk accounting, and cached chunk metadata to reflect the updated dataset dimensions and chunk validity state.

3.7. Dataset Close

Dataset close uses the flush-and-evict form of dataset flushing. All dirty chunks must first be persisted; all remaining chunk entries are then removed from the SCC before the dataset header is removed from

the dataset LRU/hash table and destroyed. This differs from an ordinary dataset or cache flush, which preserves cached entries.

The shared chunk cache must be informed whenever a dataset is closed. On receipt of this notification, the shared chunk cache must:

- Find the target instance of `H5SC_dset_header_t` in the dataset hash table in the file specific instance of `H5SC_t`.
 - Uses `H5SC_ht_dset_find()`
- Verify that the chunk LRU associated with the target instance of `H5SC_dset_header_t` is empty. Otherwise, any remaining datasets should be flushed and then have any remaining `H5SC_chunk_t` elements evicted.
- Unlink the target instance of `H5SC_dset_header_t` from the dataset hash table and dataset LRU list.
- Free the target instance of `H5SC_dset_header_t`.

3.8. Writing Data Through the SCC

When writing data through the SCC from an in-memory data buffer to a structured-chunk dataset, the SCC performs the following operations:

- Convert the dataset write request into per-chunk selection records and associate each selection with an existing or newly created `H5SC_chunk_t` entry. Chunks participating in the request are pinned while the request is active.
- Construct a stable resident-first processing order. Chunks that already contain decoded resident representations are processed before chunks requiring on-disk lookup or materialization; the underlying `sel_chunks` array is not physically reordered.
- Process resident and nonresident selections in separate batching phases. Resident chunks can be reused directly and do not require layout lookup or materialization.
- For nonresident chunks, cache the required on-disk lookup information and estimate the resident allocation required for materialization. When dataset-local resident-size observations are available, the estimate is derived from the dataset's exponential moving average and decaying high-water observation. Otherwise, the dense logical chunk size is used as the fallback estimate.
- Determine the effective active-I/O budget from unused `SCC_active_limit` headroom and bytes reclaimable under active-pressure policy. Build each batch greedily without crossing the resident/nonresident phase boundary.
 - If a single indivisible nonresident chunk cannot fit within the effective active budget, it may be processed through the oversized single-chunk admission path after SCC first reclaims all

currently reclaimable active-policy bytes; normal `SCC_active_limit` enforcement resumes after the chunk is unpinned.

- When additional space is required, invoke active-pressure reclamation. Eligible clean chunks are evicted first; eligible dirty chunks are flushed and then evicted. Active-pressure reclamation may reduce a dataset below its `min_dset_size` quiescent-retention target.
- For each write selection, reuse the resident decoded chunk when possible. Otherwise, create a new layout-specific chunk representation for an unallocated chunk or read and decode an existing on-disk representation.
- Apply the selected memory data to the resident chunk through the layout-specific gather operation, reconcile the actual resident allocation with SCC accounting, and mark the modified chunk dirty.
- Retain dirty chunks in the SCC rather than synchronously encoding and writing them as part of normal `H5SC_write()` processing. Persistent encoding, chunk-index insertion/reinsertion, and file writes occur through the applicable SCC flush path.
- After the write request completes, promote the dataset within the cache-wide dataset LRU, trim the SCC toward `SCC_quiescent_limit` using the quiescent reclamation policy, and reset the per-request I/O state while retaining reusable scratch capacity.

3.9. Reading Data Through the SCC

When reading raw data through the SCC from a structured-chunk dataset, the SCC performs the following operations:

- Convert the dataset read request into per-chunk selection records and associate each selection with an existing or newly created `H5SC_chunk_t` entry. Chunks participating in the request are pinned while the request is active.
- Construct a stable resident-first processing order. Chunks with decoded resident representations are processed before chunks requiring lookup or materialization.
- Process resident and nonresident selections in separate batching phases. Resident chunks are authoritative, including dirty chunks whose current in-memory state may differ from the persistent on-disk representation.
- For nonresident chunks, perform and cache the necessary layout lookup and estimate the resident allocation required for materialization. Dataset-local resident-size history is used when available; otherwise, the dense logical chunk size is used as the fallback estimate.
- Determine the effective active-I/O budget from unused `SCC_active_limit` headroom and bytes reclaimable under active-pressure policy, and greedily construct batches within that budget.
 - If a single indivisible nonresident chunk cannot fit within the effective active budget, it may be processed through the oversized single-chunk admission path after SCC first reclaims all

currently reclaimable active-policy bytes; normal `SCC_active_limit` enforcement resumes after the chunk is unpinned.

- When additional space is required, invoke active-pressure reclamation, evicting eligible clean chunks before flushing and evicting eligible dirty chunks.
- For each read selection, reuse an existing resident chunk when available. Otherwise, create the appropriate fill-backed representation for an undefined sparse chunk or read and decode the allocated on-disk chunk representation. Applicable filters are processed as part of structured-chunk decode.
- Scatter the selected values from the resident chunk representation into the user's memory buffer and reconcile the actual resident allocation with SCC accounting.
- After the read request completes, promote the dataset within the cache-wide dataset LRU, trim the cache toward `SCC_quiescent_limit` using quiescent-retention policy, and reset the per-request I/O state while retaining reusable scratch capacity.

3.10. Flush the Cache

`H5SC_flush()` performs a cache-wide flush-only operation. It invokes `H5SC_flush_dset(..., false)` for each dataset currently tracked by the SCC. Dirty resident chunks are persisted and become clean, but chunk and dataset entries remain cached. `H5SC_flush()` therefore does not empty the SCC and does not perform general cache teardown.

3.11. File Close

When an HDF5 file is closed, `H5SC_destroy()` is used to destroy the current SCC. This process does not flush chunks, and frees all data within the SCC using the following procedure:

- Verify that the shared chunk cache is empty.
- Free the file's instance of `H5SC_t`.

4 Detailed Component Discussions

This section provides descriptions for the various components or operations, such as the hash table and eviction candidate selection.

4.1. Chunk Tracking Through the Hash Table

To maintain consistent, constant-time lookups for chunks, a globally available hash table will be utilized through the use of UTHash. The global hash table utilized by the SCC will rely on the mechanisms provided by UTHash, through the inclusion of a single C header file. Continuing from existing HDF5 paradigms, UTHash utilizes the Jenkins hash function by default, while providing access to additional options should alternate be desirable. For each hash table item, the key, payload (i.e., the values), and the hash handle may be maintained in a single structure. The proposed `H5SC_chunk_key_t` is well suited to serve as a key when using UTHash, and provides a unique, data-focused identification for each

individual chunk². The macros for UTHash are well documented [3 and could easily be modified if need be.

4.2. Eviction

Eviction in the Shared Chunk Cache (SCC) is based on a hierarchical LRU organization consisting of a cache-wide dataset LRU and a per-dataset chunk LRU. Reclamation begins with the least-recently-used dataset and examines chunks from the least-recently-used end of that dataset's chunk LRU. The current implementation distinguishes two reclamation policies according to why cache space is being reclaimed: active-pressure reclamation and quiescent-retention trimming.

Active-pressure reclamation is performed by `H5SC__ensure_space()` when additional resident bytes must be admitted during an I/O request and the resulting cache occupancy would exceed `SCC_active_limit`. Before modifying cache topology or flushing dirty data, the SCC determines whether sufficient space is available from the cache-wide stored `reclaimable_clean_bytes` and `reclaimable_dirty_bytes` counters. If insufficient reclaimable capacity exists, the request fails without first performing partial reclamation.

When active-pressure reclamation is required, datasets are traversed from the tail of the global dataset LRU. Within each dataset, eligible clean chunks are selected first and evicted directly. If additional space is required, eligible dirty chunks are selected, flushed to persistent storage, and then evicted. The dataset LRU topology is preserved while this processing occurs; datasets are not temporarily removed or transferred to a separate exhausted-dataset list.

Active-pressure candidate selection uses the `H5SC_RECLAIM_ACTIVE` policy. Under this policy, `min_dset_size` is not treated as a hard reservation. A dataset may be reduced below its preferred minimum resident size when necessary to satisfy `SCC_active_limit`. Pinned chunks and zero-byte shell entries are not useful byte-reclamation candidates and are excluded from candidate selection.

A special case is provided for an indivisible single chunk whose estimated incremental resident size exceeds the effective active budget. Because such a chunk cannot be subdivided into a smaller batch, SCC permits a temporary oversized single-chunk admission. Before admitting the chunk, all currently reclaimable resident bytes under `H5SC_RECLAIM_ACTIVE` are reclaimed, using the normal clean-before-dirty ordering. The configured `SCC_active_limit` is not changed. Only the single oversized chunk is processed under the exception, and ordinary active-limit enforcement is restored after that chunk is unpinned.

Quiescent-retention trimming is performed by `H5SC__trim_to_quiescent_limit()` after I/O processing when `SCC_quiescent_size` exceeds `SCC_quiescent_limit`. This path uses `H5SC_RECLAIM QUIESCENT`. It follows the same general clean-before-dirty ordering, but preserves each dataset's `min_dset_size` retention target. As a result, the SCC may remain above `SCC_quiescent_limit` if the aggregate dataset retention targets prevent sufficient reclamation; in this case the trim operation returns an error.

² As a general note, UTHash does not automatically do any verification concerning the uniqueness of a given key. However, the `H5HASH_FIND` macro is provided by UTHash to allow the user to check whether a key is present within the hash table.

Candidate eligibility is centralized in `H5SC__candidate_is_reclaimable()`, while `H5SC__select_clean_candidate()` and `H5SC__select_dirty_candidate()` locate the least-recently-used eligible chunk of the requested state within a dataset. This keeps the traversal mechanism common between active and quiescent reclamation while allowing the two policies to apply different `min_dset_size` rules.

Clean chunks can be reclaimed by eviction alone. Dirty chunks must be successfully persisted through `H5SC__flush_one_chunk()` before `H5SC__evict_one_chunk()` removes them from the SCC. A failed dirty-chunk flush therefore prevents that chunk from being discarded.

4.2.1 Additional Eviction Strategies

The current implementation uses the cache-wide dataset LRU and per-dataset chunk LRUs as the ordering mechanism for reclamation. The clean/dirty candidate-selection helpers and reclamation-policy parameter provide natural extension points for evaluating alternative policies in the future.

Alternative strategies such as LFU, MRU, or weighted dataset retention would require additional policy state or metadata where necessary. Existing state that has a defined correctness role should not be repurposed for eviction ranking; in particular, `H5SC_chunk_t::chunk_counter` currently represents the chunk's pin count and is used to prevent in-flight chunks from being reclaimed.

Future configuration may expose additional eviction-policy choices or tuning parameters. Such changes should preserve the distinction between active-pressure reclamation and quiescent-retention policy unless that distinction is deliberately redesigned.

4.3. Batching

Batching in the H5SC module is implemented as a dataset-scoped, windowed execution model that separates I/O scheduling from per-chunk processing. For each dataset in a request, `H5SC__io_info_init()` constructs a dataset-owned vector of selected chunk descriptors (`sel_chunks`) that persists for the duration of the operation. A central dispatcher (the `H5SC_invoke_*` and `H5SC__invoke_*_dset_batched` routines) drives execution by processing this array in successive subranges defined by (`sel_start`, `num_sel_chunks`). Each subrange represents a batch of chunks that can be processed within current cache constraints. Batch sizing is determined by SCC memory policy. The dispatcher evaluates the incremental memory cost of bringing chunks into the cache, along with available free space (including reclaimable clean and, if necessary, dirty data), so that ordinary batches do not exceed `SCC_active_limit`. If a single indivisible chunk cannot fit within the effective active budget, the dispatcher may instead form a one-chunk oversized batch. In this case, SCC first reclaims all currently reclaimable active-policy bytes, temporarily admits and processes the chunk, and restores ordinary active-limit enforcement after the chunk is unpinned.

For nonresident chunks, the current SCC implementation estimates the resident allocation needed for admission before materialization. If dataset-local resident-size observations are available, the estimate is the maximum of the dataset's exponential moving average and decaying high-water observation. If no history is available, the dataset's dense logical chunk size is used as a fallback. These values are admission heuristics rather than formal allocation upper bounds; actual layout-reported resident allocation replaces the estimate after materialization. Lookup-based and sparse-model estimate sources are defined for future extension but are not currently used.

Selected chunks are not physically reordered in `H5SC_io_info_t::sel_chunks`. Instead, SCC builds an index vector describing a stable resident-first processing order. Chunks whose decoded objects are already resident are processed before selections requiring lookup or materialization. Relative selection order is preserved within the resident and nonresident groups. This avoids invalidating selection bookkeeping while allowing resident work to be completed before additional memory is admitted. Within each batch, lookup metadata is prepared once and reused to identify cache hits and misses and to avoid redundant layout queries. The worker functions (`H5SC_read()` and `H5SC_write()`) then operate only on the active batch, performing per-chunk operations such as index resolution, chunk creation or materialization, decoding when required, data movement through gather/scatter callbacks, and application of the requested read or write operation. Encoding and persistent write-back of dirty chunks are handled by the SCC flush paths.

This separation allows the invoke layer to manage scheduling, batching, and cache space decisions, while the worker functions remain focused on deterministic per-chunk processing. As a result, the design provides predictable memory usage, reduces repeated lookup overhead, and integrates cleanly with SCC eviction and accounting mechanisms.

4.4. Public Configuration Structure

The structure will be modeled after the `H5AC_cache_config_t` structure available in `H5ACpublic.h`. The goal of this publicly available structure is to enable users to set dataset-specific minimum retention values, eviction strategies, and global memory limits (active, quiescent, and maximum available). Default values will be included as well, serving as baseline values.

4.5. Filter Configuration and Usage

Filters for SCC-managed structured chunk datasets are configured through the dataset creation property list associated with the dataset. Structured chunk filtering uses the standard HDF5 filter pipeline while extending filter installation to operate on structured chunk section types. As noted in section 3.5, the current structured-chunk implementation disables HDF5 selection I/O when filters are configured. Filtered data is instead handled through the applicable SCC materialization, decode/encode, scatter, and gather paths.

When configuring a filtered structured chunk dataset, the following steps are performed:

- Create a dataset creation property list configured for structured chunk storage:
 - Configure the dataset layout through `H5Pset_layout()`.
 - Configure structured chunk storage through `H5Pset_struct_chunk()`.
- Install one or more filters through `H5Pset_filter2()` for the desired structured chunk section type:
 - `H5_SECTION_FIXED` for fixed structured chunk payload regions.
 - `H5_SECTION_SELECTION` for selection-managed structured chunk metadata regions.

- Configure the filter-specific parameters required by the selected filter pipeline.
- Create the dataset using the configured dataset creation property list.

Structured chunk filtering integrates directly with the SCC read/write path. During chunk write-back, SCC serializes the chunk's in-memory representation, applies the configured filter pipeline, and writes the encoded image to storage. During chunk load or materialization, SCC reads the encoded chunk image, reverses the configured filter pipeline, and reconstructs the in-memory chunk representation prior to servicing the I/O request.

Filtered SCC datasets participate in the same operational workflows as unfiltered datasets, including:

- chunk materialization,
- chunk eviction,
- batching,
- erase operations,
- extent shrink/regrow processing,
- reopen persistence,
- sparse chunk materialization,
- multidimensional chunk indexing,
- and mixed filtered/unfiltered dataset operation within the same file.

4.5.1 GZIP / DEFLATE Filtering

GZIP filtering is configured through:

- `H5Z_FILTER_DEFLATE`
- and a compression-level parameter supplied through `H5Pset_filter2()`.

GZIP filtering has been validated within SCC-managed structured chunk datasets across:

- `H5S_ALL` I/O,
- explicit hyperslab selections,
- sparse point-like access patterns represented through hyperslab selections,
- multidimensional datasets,
- extensible datasets,
- erase/rewrite workflows,

- extent shrink/regrow operations,
- reopen persistence,
- forced eviction and batching scenarios,
- and mixed-filter pipeline configurations.

Common `GZIP`-based filter pipelines include:

- `GZIP` only,
- `SHUFFLE + GZIP`,
- `FLETCHER32 + GZIP`,
- and `SHUFFLE + FLETCHER32 + GZIP`.

4.5.2 SZIP Filtering

SZIP filtering is configured through:

- `H5Z_FILTER_SZIP`
- and the coding-mode and pixels-per-block parameters supplied through `H5Pset_filter2()`.

Current SCC validation has exercised:

- nearest-neighbor (`NN`) coding mode,
- entropy-coding (`EC`) mode,
- and multiple pixels-per-block configurations.

SZIP filtering has been validated within SCC-managed structured chunk datasets across:

- full-dataset I/O,
- multi-chunk hyperslab I/O,
- partial-chunk I/O,
- sparse point-like access patterns represented through hyperslab selections,
- multidimensional datasets,
- extensible datasets,
- erase operations,
- extent shrink/regrow workflows,
- reopen persistence,

- forced eviction and batching scenarios,
- mixed-rank filtered dataset stress workloads,
- and mixed filter-pipeline configurations.

Current SZIP support applies to:

- `H5_SECTION_FIXED` structured chunk regions.

Selection-managed structured chunk metadata regions are not currently considered valid `SZIP` targets.

4.5.3 Mixed Filter Pipelines

Structured chunk datasets may apply different filter pipelines to different structured chunk section types. This allows fixed payload regions and selection-managed metadata regions to participate in separate filter pipelines within the same dataset.

Validated mixed-filter configurations include:

- `GZIP`-filtered selection metadata with `SZIP`-filtered fixed payload regions,
- mixed filtered and unfiltered datasets within the same file,
- and simultaneous operation across datasets with different filter pipelines and dimensionality.

Filtered SCC datasets participate in the same SCC accounting, residency, batching, and eviction policies as unfiltered datasets. While resident, filtered chunks may temporarily require additional transient memory during decode and re-encode operations.

5 Top Level API

These functions are called by the upper layers of the HDF5 library and serve as initial entry points to the Shared Chunk Cache (`H5SC`) package. The contents of this section largely originated in [2] and have been updated/modified where appropriate.

```
H5SC_t *H5SC_create(H5F_t *file, H5P_genplist_t *fa_plist,
H5SC__cache_config_t *config_ptr);
```

Creates a new, empty shared chunk cache. Will be called at file open time.

```
herr_t H5SC_destroy(H5SC_t *cache);
```

Destroys an already-empty shared chunk cache and releases the cache structure. This function does not flush or evict dataset chunks. Dataset close or higher-level teardown must first flush-and-evict all dataset state. Destruction fails if dataset headers, cached chunks, or accounted resident bytes remain.

```
herr_t H5SC_invoke_write(H5SC_t *cache, size_t count,
H5D_dset_io_info_t *dset_info);
```

Writes raw data through the shared chunk cache. Called by `H5D__write()`; used to initiate internal generic setup and batching through the SCC. Replaced `H5SC_write()`, which is now functionally an internal routine.

There may be datasets in the `dset_info` vector that do not support the shared chunk cache. These datasets must be ignored by the shared chunk cache. here may also be datasets that have `skip_io` set. For datasets in `dset_info` that do support the shared chunk cache, the intermediate buffers will need to be freed after the data has been written to the file.

```
herr_t H5SC_invoke_read(H5SC_t *cache, size_t count,
H5D_dset_io_info_t *dset_info);
```

Reads raw data through the shared chunk cache. Called by `H5D__read()`; used to initiate initial generic setup and batching through the SCC. Replaced `H5SC_read()`, which is now functionally an internal routine.

There may be datasets in the `dset_info` vector that do not support the shared chunk cache. These datasets must be ignored by the shared chunk cache. There may also be datasets that have `skip_io` set. These datasets must also be skipped. For datasets in `dset_info` that do support the shared chunk cache, once the data that has been scattered into the user buffer, the intermediate buffers will need to be freed using the `H5SC_chunk_evict_t` callback.

```
herr_t H5SC_read(H5SC_t *cache, H5D_dset_io_info_t *dset_info);
```

Reads raw data through the shared chunk cache. Called by `H5SC_invoke_read_dset_batched()` after initial generic setup.

```
herr_t H5SC_write(H5SC_t *cache, H5D_dset_io_info_t *dset_info);
```

Writes raw data through the shared chunk cache. Called by `H5SC__invoke_write_dset_batched()` after initial generic setup.

```
herr_t H5SC_flush(H5SC_t *cache);
```

Flushes every dirty resident chunk in the SCC to persistent storage. Successfully flushed chunks become clean but remain resident in the cache; dataset headers, chunk entries, LRU ordering, and clean resident data are retained.

```
herr_t H5SC_flush_dset(H5SC_t *cache, H5D_t *dset, bool evict);
```

Flushes dirty cached chunks associated with one dataset. When `evict_after_flush` is false, dirty chunks are persisted and retained as clean resident entries. When `evict_after_flush` is true, dirty chunks are first flushed and then all chunk entries for the dataset are evicted. The latter mode is used for dataset teardown.

```
herr_t H5SC_direct_chunk_read(H5SC_t *cache, H5D_t *dset, hsize_t *offset, void *udata, void *buf, size_t *buf_size);
```

Reads the chunk that starts at coordinates give by offset directly from disk to buf, without any decoding or conversion. First flushes that chunk if it is dirty in the cache. If `buf_size` is not large enough to read the entire chunk, nothing is read, and the needed size is returned in `*buf_size`.

These callbacks remain optional elements of the generic SCC layout interface and are not currently supplied by the structured-chunk layout.

```
herr_t H5SC_direct_chunk_write(H5SC_t *cache, H5D_t *dset, hsize_t *offset, void *udata, const void *buf);
```

Writes the chunk that starts at coordinates given by offset directly from buf to disk, without any encoding or conversion. First invalidates and evicts that chunk from cache if it is present. **These callbacks remain optional elements of the generic SCC layout interface and are not currently supplied by the structured-chunk layout.**

```
H5S_t *H5SC_get_defined(H5SC_t *cache, H5D_t *dset, const H5S_t *file_space);
```

Returns a `dataspace` selection describing the elements that are currently defined within the requested structured-chunk dataset selection. The operation queries resident chunks when available and may decode defined-value metadata for nonresident chunks without intentionally retaining those temporary chunk representations in the SCC.

```
herr_t H5SC_erase(H5SC_t *cache, H5D_t *dset, const H5S_t
*file_space);
```

Causes the elements selected in `file_space` to become undefined in `dset`. If `dset` does not support tracking defined elements, returns an error.

```
herr_t H5SC_set_extent_notify(H5SC_t *cache, H5D_t *dset, const
hsize_t *old_dims);
```

Called after `H5Dset_extent()` has been called for a dataset, so the cache can recompute chunk indices, delete chunks, clear unused sections of chunks, etc.

```
herr_t H5SC_validate_config(const H5SC__cache_config_t *config_ptr);
```

Runs a sanity check on the contents of the supplied instance of `H5SC__cache_config_t`. Related to the integration of File Access Property List (FAPL) support.

```
H5SC_dset_header_t *H5SC_dset_create_header(H5SC_t *cache, haddr_t
addr, size_t max_chunk_size);
```

Allocates and initializes a `H5SC_dset_header_t` structure with the specified dataset address. Does not add this structure to the hash table or appropriate LRU list. Necessary for `H5D__create` and `H5D__open_oid()` to operate properly. If the default minimum dataset size (10 MB) is greater than the quiescent limit for the SCC, the minimum dataset size is set to be two times the max chunk size. This necessary to prevent odd eviction behavior when the quiescent limit is small (mostly useful for testing).

```
herr_t H5SC_dset_destroy_header(H5SC_t *cache, H5D_t *dset,
H5SC_dset_header_t *dset_hdr);
```

Frees any remaining chunks, persistent `io_info`, and frees the dataset header structure. Assumes that the dataset has been freed from the dataset LRU and dataset hash table.

```
herr_t H5SC_dset_is_empty(H5SC_dset_header_t *dset_hdr, bool
*is_empty);
```

Helper function that inspects a dataset header to ensure no further chunk data is present. Due to how `H5SC_dset_header_t` is defined in the H5SC module, this is necessary for instances where a different module must interact with a field in this structure (such as within `H5D_close()`).

```
haddr_t H5SC_dset_get_addr(H5SC_dset_header_t *dset_hdr);
```

Function used to get the address embedded within a dataset header structure. Due to how `H5SC_dset_header_t` is defined, this is necessary for instances where a different module must interact with a field in this structure (such as within `H5D_close()`).

```
herr_t H5Pset_scc_config(hid_t plist_id, H5SC__cache_config_t
*config_ptr);
```

Function used to set the shared chunk cache configuration data in the provided File Access Property List (FAPL). Currently only supports setting non-zero values for the quiescent and active size limits.

```
herr_t H5Pget_scc_config(hid_t plist_id, H5SC__cache_config_t
*config /*out*/);
```

Function used to retrieve the current (or default) shared chunk cache configuration from the provided File Access Property List (FAPL). Note that this function will return an error if `config` is `NULL` or if `config_ptr->version` specifies an unknown version of `H5AC_cache_config_t`. Expect to use `H5SC__CURR_SCC_VERSION` when defining the `H5SC__cache_config_t` structure.

6 SCC Internal Functions

Functions that are internal to the SCC operations. Functions are presented in categories based on their primary functionality.

6.1. Hash table specific functions

```
void H5SC__hash_init(H5SC_t *cache);
```

Initializes the SCC chunk and dataset hash-table heads to an empty state. This routine establishes hash-table state for a newly created SCC but does not allocate entries.

```
void H5SC__reset_hash_tables(H5SC_t *cache);
```

Resets the SCC hash-table heads after all entries have already been removed. The routine unlinks hash-table state and sets both heads to `NULL`; it does not free or otherwise destroy chunk or dataset payload objects.

```
H5SC_chunk_t *H5SC__ht_chunk_find(H5SC_t *cache, const
H5SC_chunk_key_t *chk_key);
```

Helper function used to lookup a chunk record by its unique (per file) `H5SC_chunk_key_t` and returns the stored pointer.

```
herr_t H5SC__ht_chunk_insert(H5SC_t *cache, H5SC_chunk_t *chk);
```

Helper function that inserts a chunk record into the chunk hash table. A unique `H5SC_chunk_key_t` is used as the key (unique for each file).

```
herr_t H5SC__ht_chunk_delete(H5SC_t *cache, const H5SC_chunk_key_t *chk_key);
```

Helper function that unlinks (removes) a chunk from the chunk hash table by address. This does not free the unlinked node; this is left to the caller.

```
H5SC_dset_header_t *H5SC__ht_dset_find(H5SC_t *cache, haddr_t addr);
```

Helper function used to lookup a dataset header record by its unique (per file) `haddr_t` and returns the stored pointer.

```
herr_t H5SC__ht_dset_insert(H5SC_t *cache, H5SC_dset_header_t *dset_hdr);
```

Helper function that inserts a dataset header record into the dataset hash table. The dataset object address header is used as the key (unique for each file).

```
herr_t H5SC__ht_dset_delete(H5SC_t *cache, haddr_t addr);
```

Helper function that unlinks (removes) a dataset header from the dataset header hash table by address. This does not free the unlinked node; this is left to the caller.

6.2. DLL specific functions

```
herr_t H5SC__dset_lru_prepend(H5SC_t *cache, struct H5SC_dset_header_t *dset_hdr);
```

Helper function used to insert a dataset header from the dataset LRU list. Increments the SCC quiescent size. It is the caller's responsibility to decrement the chunk counter (if appropriate).

```
herr_t H5SC__dset_lru_promote(H5SC_t *cache, H5SC_dset_header_t *dset_hdr);
```

Helper function used to move a dataset header to the head of the dataset LRU list.

```
herr_t H5SC__dset_lru_remove(H5SC_t *cache, struct H5SC_dset_header_t *dset_hdr);
```

Helper function used to remove a dataset header from the dataset LRU list. Decrements the SCC quiescent size. It is the caller's responsibility to decrement the chunk counter (if appropriate).

```
herr_t H5SC__chunk_lru_prepend(H5SC_t *cache, struct
H5SC_dset_header_t *dset_hdr, struct H5SC_chunk_t *chunk);
```

Helper function used to insert a chunk at the head of the appropriate dataset's LRU list. Increments the dataset size. It is the caller's responsibility to increment the chunk counter (if appropriate). Does not interact with SCC quiescent size; this is left as a responsibility of the caller and may not always be desirable.

```
herr_t H5SC__chunk_lru_remove(H5SC_t *cache, struct
H5SC_dset_header_t *dset_hdr, struct H5SC_chunk_t *chunk);
```

Helper function used to remove a chunk from the appropriate dataset's LRU list. Decrements the dataset size. It is the caller's responsibility to increment the chunk counter (if appropriate). Does not interact with SCC quiescent size; this is left as a responsibility of the caller and may not always be desirable.

6.3. H5SC_chunk_t specific functions

```
static herr_t H5SC__compute_logical_chunk_index(unsigned ndims,
const hsize_t *dset_dims, const hsize_t *chunk_dims, const hsize_t
*elem_coord, hsize_t *log_chk_idx);
```

Helper function used to compute the logical chunk coordinate. This value is unique within a dataset, but not across individual datasets.

```
static herr_t H5SC__compute_chunk_key(haddr_t const
*dset_object_header_addr, const hsize_t *log_chk_coord,
H5SC_chunk_key_t *chunk_key);
```

Helper function used to compute the chunk key associated with a dataset object header and chunk logical coordinate pair.

```
static H5SC_chunk_t *H5SC__make_chunk(H5SC_chunk_key_t key, size_t
cached_sz, size_t counter, bool pio);
```

Function that initializes an `H5SC_chunk_t` instance for a specific `H5SC_chunk_key_t`. This does not add the newly created chunk to the hash table or appropriate LRU list.

```
static herr_t H5SC__chunk_teardown_resident(H5D_t *dset,
H5SC_chunk_t *chunk);
```

Helper function used to teardown a resident decoded chunk object held by the SCC. Utilizes the evict callback function (`H5D__struct_chunk_evict()`).

6.4. I/O specific functions

```
static inline void H5SC__io_sel_chunk_init(H5SC_io_sel_chunk_t *c);
```

Helper function used to initialize an instance of `H5SC_io_sel_chunk_t` with safe default values.

```
static herr_t H5SC__io_info_init(H5SC_t *cache, size_t count,
H5D_dset_io_info_t *dset_info);
```

Helper function used to initialize `sc_io_info` for an I/O operation, including computing the per-chunk memory and file data spaces. Additionally, this function allocates and fills in the `sc_io_info->sel_chunks` array, which is the array of chunks that are involved in this I/O operation. Chunks not yet in the cache that are present in this I/O operation have a placeholder created, which is treated as a non-resident chunk until it has been processed.

```
static herr_t H5SC__erase_io_info_init(H5SC_t *cache, H5D_t *dset,
const H5S_t *file_space);
```

Helper function derived from `H5SC__io_info_init()` that is called by `H5SC_erase()` to identify file space information necessary for the erase operation.

```
static herr_t H5SC__io_info_reset(H5SC_io_info_t *sc_io_info);
```

Helper function used to reset the contents of an `sc_io_info` structure to default values for later reuse.

```
static herr_t H5SC__io_info_term(H5SC_io_info_t *sc_io_info);
```

Helper function used to free all memory referenced by an `sc_io_info` structure. Does not free the `sc_io_info` structure itself.

```
static herr_t H5SC__invoke_read_dset_batched(H5SC_t *cache,
H5D_dset_io_info_t *dset_info, H5SC_dset_header_t *dset_hdr);
```

Helper function that is responsible for determining whether batching is necessary to complete the requested read operation. Currently offers two modes; a chunk-by-chunk mode that processes exactly one chunk per call to `H5SC_read()` and size-restricted batching mode. The chunk-by-chunk mode has primarily been used as a debugging tool during development. The size-restricted batching mode operates as a greedy algorithm and will attempt to process as many chunks as possible in each batch based on the available memory (SCC active limit) and the amount of space that can be freed, as computed by other helper functions. Batch sizes need not be uniform.

```
static herr_t H5SC__invoke_write_dset_batched(H5SC_t *cache,
H5D_dset_io_info_t *dset_info, H5SC_dset_header_t *dset_hdr);
```

Helper function that is responsible for determining whether batching is necessary to complete the requested write operation. Currently offers two modes; a chunk-by-chunk mode that processes exactly one chunk per call to `H5SC_write()` and size-restricted batching mode. The chunk-by-chunk mode has primarily been used as a debugging tool during development. The size-restricted batching mode operates as a greedy algorithm and will attempt to process as many chunks as possible in each batch based on the available memory (SCC active limit) and the amount of space that can be freed, as computed by other helper functions. Batch sizes need not be uniform.

```
static herr_t H5SC__lookup_cache_misses(H5D_t *dset, H5SC_io_info_t
*sc_io_info);
```

Helper function that uses the lookup callback (`H5D__struct_chunk_lookup()`) to lookup the on disk chunk addresses and chunk sizes that are not currently a resident of the SCC. Also utilized during batching.

```
static herr_t H5SC__selection_io_info_init(H5D_t *dset, const H5S_t
*file_space, H5SC_io_info_t *sc_io_info);
```

Common selection-decomposition helper used to convert a supported dataset file-space selection into per-chunk SCC selection records. The helper supports `H5S_SEL_ALL` and hyperslab selections, treats empty selections as no-ops, and rejects point selections.

```
static inline H5SC_io_sel_chunk_t *H5SC__io_sel_at(const
H5SC_io_info_t *io, size_t relative_idx);
```

Resolves a relative selected-chunk index to the corresponding `H5SC_io_sel_chunk_t`. The helper accounts for either contiguous `sel_chunks` traversal or the active `sel_order` view used by resident-first batching.

```
static void H5SC__io_scratch_free_storage(H5SC_io_scratch_t
*scratch);
```

Releases the backing allocations owned directly by an `H5SC_io_scratch_t` without releasing objects referenced through its non-owning pointer vectors. The structure is reset to an empty state after its owned storage is released.

```
static herr_t H5SC__io_scratch_ensure(H5SC_io_info_t *sc_io_info,
size_t needed);
```

Ensures that the dataset-owned reusable I/O scratch workspace has capacity for at least needed entries. Scratch storage is allocated lazily and grown geometrically; existing storage remains valid if replacement allocation fails.

```
static herr_t H5SC__build_resident_first_order(H5SC_io_info_t
*sc_io_info);
```

Builds the stable processing-order vector used by read/write batching so that selected chunks with decoded resident objects are processed before nonresident chunks requiring lookup or materialization. Relative order is retained within both groups.

```
static herr_t H5SC__get_defined_chunk(H5D_t *dset, const
H5SC_io_sel_chunk_t *sel, H5SC_chunk_t *cached_chk, bool filtered,
bool partial_bound_chunks_different_encoding, H5S_t
**chunk_defined);
```

Determines which selected elements of one logical chunk are currently defined. Resident SCC state is authoritative; otherwise the helper performs a cache-neutral on-disk lookup and, when required, decodes only defined-value metadata without adding the temporary representation to the SCC.

```
static herr_t H5SC__merge_defined_chunk_selection(H5S_t *defined,
const H5S_t *chunk_defined, const H5SC_io_sel_chunk_t *sel, unsigned
ndims, const hsize_t *dset_dims, const hsize_t *chunk_dims);
```

Transforms a chunk-local defined-value selection into dataset coordinates and merges it into the accumulating dataspace returned by `H5SC_get_defined()`.

6.5. Eviction specific functions

```
static herr_t H5SC__flush_one_chunk(H5SC_t *cache, H5D_t *dset,
H5SC_dset_header_t *dset_hdr, H5SC_chunk_t *chk);
```

Helper function used to persist one dirty resident chunk. The operation performs any required layout query and lookup, non-destructively encodes the resident representation, updates or allocates the corresponding on-disk chunk index entry through the layout insert callback, and writes the encoded representation using `H5F_block_write()`. A successfully flushed chunk becomes clean but remains resident unless the caller subsequently evicts it.

```
static herr_t H5SC__evict_one_chunk(H5SC_t *cache, H5D_t *dset,
H5SC_dset_header_t *dset_hdr, H5SC_chunk_t *chk);
```

Helper function used to remove one eligible chunk from the SCC. The routine releases any resident layout-specific representation, removes the chunk from its dataset LRU and the SCC chunk hash table, updates associated cache accounting, and frees the `H5SC_chunk_t` structure. Dirty chunks must be successfully flushed before this function is used to discard their current data.

```
static bool H5SC__candidate_is_reclaimable(const H5SC_dset_header_t
*dset_hdr, const H5SC_chunk_t *chunk, H5SC_reclaim_policy_t policy);
```

Common eligibility helper used by clean and dirty candidate selection. Rejects chunks that cannot currently be reclaimed, including pinned chunks and entries that do not contribute resident bytes.

The supplied reclamation policy determines whether the dataset's `min_dset_size` retention target must be preserved. `H5SC_RECLAIM_ACTIVE` permits reclamation below `min_dset_size`, while `H5SC_RECLAIM_QUIESCENT` preserves that target.

```
static bool H5SC__select_clean_candidate(H5SC_dset_header_t
*dset_hdr, H5SC_reclaim_policy_t policy, H5SC_chunk_t **candidate);
```

Selects the least-recently-used eligible clean resident chunk from the supplied dataset under the requested reclamation policy. Clean candidates may be evicted without first being flushed.

```
static bool H5SC__select_dirty_candidate(H5SC_dset_header_t
*dset_hdr, H5SC_reclaim_policy_t policy, H5SC_chunk_t **candidate);
```

Selects the least-recently-used eligible dirty resident chunk from the supplied dataset under the requested reclamation policy. Dirty candidates must be successfully flushed before eviction.

```
static inline size_t H5SC__cached_active_reclaimable(const H5SC_t
*cache);
```

Returns the cache-wide resident bytes currently considered reclaimable under active-pressure policy from the stored clean and dirty reclaimability counters. This avoids performing a full SCC traversal merely to determine whether an active-limit admission request is feasible.

```
static inline size_t H5SC__effective_budget(const H5SC_t *cache);
```

Computes the effective amount of memory available to the invoke-layer batching logic from unused active-limit headroom together with resident bytes reclaimable under active-pressure policy. When `H5SC_DO_SANITY_CHECKS` is enabled, additional verification helpers recompute reclaimability and confirm that the selected clean or dirty candidate agrees with the normal policy-aware selection routines:

```
static size_t H5SC__calc_reclaimable_bytes(const H5SC_t *cache,
H5SC_reclaim_policy_t policy);
```

Sanity-check helper that recomputes the total number of bytes reclaimable under the supplied reclamation policy by traversing the current SCC state. This value is calculated independently of the cache's stored reclaimability counters and is used to verify that incremental reclaimability accounting remains consistent with the actual chunk state. This function is compiled only when `H5SC_DO_SANITY_CHECKS` is enabled.

```
static herr_t H5SC__verify_cached_reclaimability(const H5SC_t
*cache);
```

Sanity-check helper that compares the SCC's cached clean and dirty reclaimability counters with independently recomputed values derived from the chunks currently resident in the cache. Returns failure when the stored accounting does not agree with the cache topology and chunk state. This function is intended for invariant checking rather than normal reclamation processing.

```
static herr_t H5SC__verify_clean_candidate(H5SC_dset_header_t
*dset_hdr, H5SC_chunk_t *candidate, H5SC_reclaim_policy_t policy);
```

Sanity-check helper used to verify that the clean chunk selected by the normal candidate-selection path is the expected least-recently-used eligible clean candidate under the supplied reclamation policy. This provides an independent check of clean-candidate ordering and eligibility rules.

```
static herr_t H5SC__verify_dirty_candidate(H5SC_dset_header_t
*dset_hdr, H5SC_chunk_t *candidate, H5SC_reclaim_policy_t policy);
```

Sanity-check helper used to verify that the dirty chunk selected by the normal candidate-selection path is the expected least-recently-used eligible dirty candidate under the supplied reclamation policy. This provides an independent check of dirty-candidate ordering and eligibility rules.

6.6. Size accounting functions

```
static herr_t H5SC__ensure_space(H5SC_t *cache, size_t bytes_needed,
bool allow_oversized_single_chunk);
```

Ensures that the requested incremental resident bytes can be admitted under the active SCC policy. For ordinary admission, the function verifies that sufficient active-policy capacity can be obtained and reclaims eligible clean chunks before flushing and evicting eligible dirty chunks. If `allow_oversized_single_chunk` is `true` and the request represents one indivisible chunk whose estimated size exceeds the effective active budget, the function may invoke the oversized admission exception. In that case, all currently reclaimable active-policy bytes are reclaimed before the chunk is admitted, even though the resulting temporary working set may exceed `SCC_active_limit`. The configured active limit itself is not modified.

```
static inline bool H5SC__active_reclaim_required(const H5SC_t
*cache, size_t bytes_needed, bool oversized);
```

Determines whether active-pressure reclamation should continue while preparing an SCC admission. For ordinary admission, reclamation continues while the requested incremental allocation would exceed `SCC_active_limit`. For an oversized single-chunk admission, reclamation instead continues until no clean or dirty resident bytes remain reclaimable under `H5SC_RECLAIM_ACTIVE`.

```
herr_t H5SC__chunk_update_cached_size(H5SC_t *cache,
H5SC_dset_header_t *dset_hdr, struct H5SC_chunk_t *chunk, size_t
new_size);
```

Helper function used to update the chunk size stored within an instance of a resident non-placeholder chunk.

```
static size_t H5SC__calc_inc_for_chunk(const H5D_t *dset, const
H5SC_chunk_t *chk, size_t desired);
```

Helper function that computes the bytes necessary to materialize/resize a chunk during batching.

```
static herr_t H5SC__trim_to_quiescent_limit(H5SC_t *cache);
```

Performs steady-state reclamation after I/O until SCC_quiescent_size <= SCC_quiescent_limit. Candidate selection uses H5SC_RECLAIM QUIESCENT, preserves dataset min_dset_size retention targets, prefers clean eviction before dirty flush-and-evict, and fails if the configured quiescent target cannot be reached from the eligible candidates.

```
static herr_t H5SC__account_dset_size_change(H5SC_t *cache,
H5SC_dset_header_t *dset_hdr, size_t old_size, size_t new_size);
```

Helper function used to apply a change in dataset size to the SCC quiescent size.

```
static herr_t H5SC__account_chunk_link_change(H5SC_t *cache,
H5SC_dset_header_t *dset_hdr, size_t old_dset_size);
```

Helper function that is used to update the current in-cache dataset size after an appropriate chunk DLL operation (insert/remove/resize). Also responsible for calling H5SC__account_dset_size_change() to update the quiescent size.

```
static inline H5SC_reclaim_contrib_t
H5SC__chunk_reclaim_contribution(const H5SC_dset_header_t *dset_hdr,
const H5SC_chunk_t *chunk);
```

Computes the clean and dirty reclaimable-byte contribution associated with a chunk's current resident, pin, and dirty state under active-pressure accounting.

```
static inline herr_t H5SC__apply_reclaim_transition(H5SC_t *cache,
H5SC_dset_header_t *dset_hdr, H5SC_reclaim_contrib_t before,
H5SC_reclaim_contrib_t after);
```

Applies the difference between a chunk's previous and new reclaimability contributions to the per-dataset and cache-wide stored clean/dirty reclaimability counters.

```
static inline herr_t H5SC__chunk_set_dirty(H5SC_t *cache,
H5SC_dset_header_t *dset_hdr, H5SC_chunk_t *chunk, bool dirty);
```

Changes a chunk's dirty state while maintaining the corresponding reclaimability accounting. Dirty-state transitions should be routed through this helper while a chunk is tracked by the SCC.

```
static H5SC__chunk_pin(H5SC_t *cache, H5SC_dset_header_t *dset_hdr,
H5SC_chunk_t *chunk);
```

Increment a chunk's SCC pin count while updating its reclaimability contribution. Pinned chunks are protected from reclamation, so pin transitions must update both chunk state and cached reclaimable-byte accounting.

```
static inline herr_t H5SC__chunk_unpin(H5SC_t *cache,
H5SC_dset_header_t *dset_hdr, H5SC_chunk_t *chunk);
```

Decrement a chunk's SCC pin count while updating its reclaimability contribution. Pinned chunks are protected from reclamation, so pin transitions must update both chunk state and cached reclaimable-byte accounting.

```
static herr_t H5SC__estimate_resident_size(const H5SC_dset_header_t
*dset_hdr, const H5D_t *dset, const H5SC_io_sel_chunk_t *sel, bool
is_write, size_t write_bytes, H5SC_size_est_source_t *source, size_t
*estimate);
```

Produces the resident-allocation estimate used when admitting a nonresident chunk into a batch. The current implementation uses dataset-local resident-size history when available and otherwise falls back to the dense logical chunk size; lookup and sparse-model estimate sources are reserved for future use.

```
static void
H5SC__update_resident_estimate_history(H5SC_dset_header_t *dset_hdr,
size_t actual);
```

Updates the dataset-local resident-size history after a successful final resident allocation has been observed, maintaining the exponential moving average, decaying high-water observation, and sample count used by subsequent estimates.

6.7. Extensible dimension-related functions

```
static bool H5SC__chunk_outside_extent(const H5D_t *dset, const
hsize_t *chunk_dims, const hsize_t *scaled);
```

Helper function used to determine whether a chunk lies completely outside a dataset's current extent.

```
static bool H5SC__chunk_is_partial_bound(unsigned ndims, const
hsize_t *chunk_dims, const hsize_t *scaled, const hsize_t *dims);
```

Helper function that is used to determine whether a chunk is considered to have a partial boundary (otherwise called an edge chunk) with respect to dims.

```
static bool H5SC__chunk_needs_erase(const H5D_t *dset, const hsize_t
*chunk_dims, const hsize_t *scaled, const hsize_t *old_dims);
```

Helper function used to determine whether a surviving chunk overlaps a newly invalidated region after extent shrinkage (which would require erasing values).

```
static herr_t H5SC__rekey_dset_chunks_after_extent_change(H5SC_t
*cache, H5D_t *dset, H5SC_dset_header_t *dset_hdr, const hsize_t
*chunk_dims);
```

Helper function used to recompute logical chunk coordinates and chunk keys (`H5SC_chunk_key_t`) for all chunks in a dataset after an extent change that alters the dataset's chunk grid.

```
static herr_t H5SC__erase_chunk_invalid_extent_region(H5D_t *dset,
const hsize_t *chunk_dims, const hsize_t *old_dims, H5SC_chunk_t
*chk, size_t *nbytes, size_t *alloc_size, bool *delete_chunk);
```

Helper function used to erase invalid values from a resident structured chunk after a dataset extent shrink. This function also erases any newly invalidated on-disk values using the SCC callback functions.

```
static herr_t H5SC__for_each_scaled_coord_outside_extent(H5SC_t
*cache, H5D_t *dset, unsigned ndims, const hsize_t *old_nchunks,
const hsize_t *new_nchunks);
```

Helper function used to iterate over the scaled chunk coordinates that were present in the old chunk grid but are now outside the new chunk grid after an extent shrink.

```
static herr_t H5SC__prune_one_scaled_coord_outside_extent(H5SC_t
*cache, H5D_t *dset, const hsize_t *scaled);
```

Helper function used to delete a single structured chunk coordinate that lies outside the dataset's new extent.

```
static herr_t H5SC__lookup_chunk_for_prune(H5D_t *dset, const
hsize_t *scaled, haddr_t *addr, hsize_t *disk_nbytes, bool *exists);
```

Helper function used to query on-disk storage for a chunk during extent-prune cleanup.

6.8. Statistics-specific functions

The SCC maintains a set of lightweight counters unconditionally and provides additional diagnostic statistics when selected compile-time options are enabled. Functions described below that are controlled by a compile-time option are not present in builds where that option is disabled.

`H5SC_COLLECT_ESTIMATE_STATS` enables detailed comparison of predicted resident allocation sizes against the final observed resident allocations. This instrumentation records estimate-source usage, total estimated and actual bytes, over- and under-estimation frequency and magnitude, and write-growth observations.

`H5SC_ENABLE_STAT_DUMPS` enables calculation and formatted output of SCC statistics, including cache hit/miss rates and the statistics dump produced during diagnostic use or cache destruction. These facilities are intended for development, validation, and performance characterization rather than correctness of normal SCC operation.

```
static void H5SC__stats_reset(H5SC_t *cache);
```

Responsible for resetting the counters present within an instance of `H5SC_stats_t`. Used when an instance of `H5SC_t` is created or destroyed.

```
static void H5SC__stats_record_lookup(H5SC_t *cache, hbool_t hit);
```

Used to record each time a chunk is looked up within the SCC (i.e., using `H5SC__ht_chunk_find()`). This count is used as the denominator when computing hit/miss rates.

```
static void H5SC__stats_record_insert(H5SC_t *cache);
```

Used to count the number of times chunks are inserted into the chunk-focused hash table. This does not reflect when a chunk is linked/relinked within a DLL.

```
static void H5SC__stats_record_delete(H5SC_t *cache);
```

Increases the count of chunks that have been removed from the SCC in situations where a chunk removal is logically distinct from eviction. It is currently used when a chunk is removed from the hash table but may be adapted later for tracking changes in extent or other interesting types of removals.

```
static void H5SC__stats_record_chunk_flush(H5SC_t *cache);
```

Used to count the number of individual chunk flushes within the SCC.

```
static void H5SC__stats_record_dset_flush(H5SC_t *cache);
```

Used to count the number of individual dset flushes within the SCC.

```
static void H5SC__stats_record_batch_read(H5SC_t *cache);
```

Used to count the number of individual read batches that have been processed within the SCC.

```
static void H5SC__stats_record_batch_write(H5SC_t *cache);
```

Used to count the number of individual write batches that have been processed within the SCC.

```
static void H5SC__stats_record_batch_read_len(H5SC_t *cache, size_t batch_len);
```

Used to record the size of the most recent read batch and update the maximum observed batch length (if applicable).

```
static void H5SC__stats_record_batch_write_len(H5SC_t *cache, size_t batch_len);
```

Used to record the size of the most recent write batch and update the maximum observed batch length (if applicable).

```
static void H5SC__stats_record_eviction(H5SC_t *cache);
```

Used to count the times chunk eviction is triggered within the SCC.

```
static void H5SC__report_oversized_admission(H5SC_t *cache, size_t estimated_bytes);
```

Records use of the oversized single-chunk admission path. The helper increments `scc_oversized_admission_count`, updates the maximum admitted estimated size and maximum amount by which an estimate exceeded `SCC_active_limit`, and emits a diagnostic warning indicating that the configured active limit was temporarily exceeded. An excess of zero is possible when the exception is triggered because the chunk exceeds the current effective budget even though its estimated size does not itself exceed `SCC_active_limit`.

```
#if defined(H5SC_COLLECT_ESTIMATE_STATS) &&
(H5SC_COLLECT_ESTIMATE_STATS + 0)
    static void H5SC__stats_record_size_estimate(H5SC_t *cache,
        H5SC_size_est_source_t source, size_t estimate, size_t actual);
#endif
```

Records the accuracy of a completed resident-size estimate after the selected chunk has been materialized or otherwise reached its final resident allocation for the operation. The routine increments the total estimate count and accumulates the estimated and actual resident byte totals. It also attributes the observation to the `H5SC_size_est_source_t` mechanism that produced the

estimate. When the estimate exceeds the final resident allocation, the helper records an overestimate, accumulates the number of excess estimated bytes, and updates the maximum observed overestimate. When the final allocation exceeds the estimate, the corresponding underestimate count, cumulative underestimated bytes, and maximum observed underestimate are updated. Exact estimates contribute to the total and per-source counts without incrementing either error category.

This helper is compiled only when `H5SC_COLLECT_ESTIMATE_STATS` is enabled and is diagnostic only; it does not alter cache admission decisions, resident-size history, or authoritative SCC size accounting.

```
#if defined(H5SC_ENABLE_STAT_DUMPS) && (H5SC_ENABLE_STAT_DUMPS + 0)
static double H5SC__stats_get_hit_rate(const H5SC_t *cache);
```

Returns the SCC hit rate for a cache instance as a floating point ratio (expected to be between 0.0 and 1.0). In this context, a hit indicates that a requested chunk lookup (within the SCC) successfully resolved to a resident non-placeholder chunk that was present within the cache.

```
static double H5SC__stats_get_miss_rate(const H5SC_t *cache);
```

Returns the SCC miss rate for a cache instance as a floating point ratio (expected to be between 0.0 and 1.0). In this context, a miss indicates that a requested chunk lookup (within the SCC) did not resolve to a resident non-placeholder chunk that was present within the cache.

```
static herr_t H5SC__stats_dump(const H5SC_t *cache);
```

Function responsible for printing statistics tracked by the shared chunk cache. These statistics are stored in an `H5SC_stats_t` structure which is stored in `H5SC_t`.

```
#endif
```

7 Layout Callbacks

These are the callback functions that individual layout types implement in order to enable the shared chunk cache to perform these operations in a layout-agnostic manner. Some of these functions accept an `hsize_t *` parameter called `scaled`. This is an array containing the scaled coordinates of the chunk, where the coordinates are divided by the chunk dimensions so that adjacent chunks differ in their scaled coordinates by a value of one. In addition, in order for the shared chunk cache to calculate the logical locations of the chunks, the chunk dimensions will be promoted to the `H5D_shared_t` struct and made available for all layout types where it is valid (contiguous datasets will simply use the dataset dimensions). The layout will also need to specify whether it uses the legacy vlen/reference storage where data is stored outside the dataset, or whether the variable length data is stored in a separate section of the chunk. The contents of this section largely originated in [2] and have been updated/modified where appropriate.

```
typedef herr_t (*H5SC_chunk_lookup_t)(H5D_t *dset, size_t count, const
hsize_t *scaled[] /*in*/, haddr_t *addr[] /*out*/, hsize_t *size[]
/*out*/, hsize_t *defined_values_size[] /*out*/, size_t *size_hint[]
/*out*/, size_t *defined_values_size_hint[] /*out*/, void **udata
/*out*/);
```

Looks up the on-disk representation associated with count chunks identified by their scaled coordinates. For each chunk, the callback returns the current on-disk address and encoded size, together with information describing the encoded defined-value metadata and allocation hints that may be used by subsequent read/decode processing. `defined_values_size` reports the encoded byte size of the defined-value metadata. A value of zero indicates that all values in the chunk are defined. `size_hint` provides the layout's suggested allocation size for the encoded chunk buffer used by the read/decode path, while `defined_values_size_hint` provides the corresponding allocation hint when only defined-value metadata is required. These hints describe encoded-buffer requirements and must not be interpreted as complete decoded resident-allocation estimates unless a layout explicitly documents such behavior.

The callback may also return layout-specific state through `udata`. The SCC treats this object as opaque and may pass it to subsequent decode, selection-I/O, vector-I/O, or other applicable layout callbacks. Ownership and release semantics for the returned object are defined by the layout implementation and SCC callback contract; callers should not assume that the object can always be released with a particular generic allocator. If no persistent representation exists for a requested chunk, the callback returns an undefined address for that chunk. Any layout-specific `udata` produced for such a miss must be released or transferred according to the layout's ownership rules.

```
typedef herr_t (*H5SC_chunk_decode_t)(H5D_t *dset, size_t *nbytes
/*in,out*/, size_t *alloc_size /*in,out*/, bool partial_bound, void
**chunk /*in,out*/, void *udata);
```

Decompresses/decodes the chunk from file format to memory cache format if necessary. Reallocs chunk buffer if necessary. On entry, `nbytes` is the number of bytes used in the chunk buffer. On exit, it shall be set to the total number of bytes used (not allocated) across all buffers for this chunk. On entry, `alloc_size` is the size of the chunk buffer. On exit, it shall be set to the total number of bytes allocated across all buffers for this chunk. Optional, if not present, chunk is the same in cache as on disk. `partial_bound` is true if the chunk was encoded with `partial_bound` set to true. If the dataset reported `partial_bound_chunks_different_encoding` as false, the setting of `partial_bound` is undefined.

```
typedef herr_t (*H5SC_chunk_decode_defined_values_t)(H5D_t *dset,
size_t *nbytes /*in,out*/, size_t *alloc_size /*in,out*/, bool
partial_bound, void **chunk /*in,out*/, void *udata);
```

The same as `H5SC_chunk_decode_t` but only decodes the defined values. Optional, if not present, the entire chunk must always be decoded.

```
typedef herr_t (*H5SC_new_chunk_t)(H5D_t *dset, bool fill, size_t
*nbytes /*out*/, size_t *buf_size /*out*/, void **chunk /*chunk*/,
void **udata /*out*/);
```

Creates a new empty chunk and a new empty udata. Does not insert into on disk chunk index. If fill is true, writes the fill value to the chunk (unless this is a sparse chunk). The number of bytes used is returned in **nbytes* and the size of the chunk buffer is returned in **buf_size*.

```
typedef herr_t (*H5SC_chunk_condense_t)(H5D_t *dset, size_t *nbytes
/*in, out*/, void **chunk /*in, out*/, void *udata);
```

Reallocates buffers as necessary so the total allocated size of buffers for the chunk (*alloc_size*) is equal to the total number of bytes used (*nbytes*). Optional, if not present the chunk cache will be more likely to evict chunks if there is wasted space in the buffers.

```
typedef herr_t (*H5SC_chunk_encode_t)(H5D_t *dset, hsize_t *write_size
/*out*/, hsize_t *write_buf_alloc /*out*/, bool partial_bound, const
void *chunk, void *udata, void **write_buf /*out*/);
```

Produces the encoded on-disk representation of a resident chunk without modifying the resident decoded chunk object. The callback applies any layout-specific serialization, compression, or filter processing required to transform the cached representation into the form that will be written to persistent storage. *write_size* receives the number of encoded bytes that must be written. If a separate encoded buffer must be allocated, *write_buf* receives that buffer and *write_buf_alloc* reports the size of its allocation. When the existing chunk representation can be written directly without producing a separate encoded buffer, *write_buf* may remain NULL according to the layout contract. *partial_bound* indicates whether the chunk is currently a partial-edge chunk. This value is significant only when the layout reports through *layout_query* that partial-boundary chunks may use a different encoding. The callback must preserve the resident decoded chunk representation. Destructive or in-place encoding is handled separately through *H5SC_chunk_encode_in_place_t*.

```
typedef herr_t (*H5SC_chunk_encode_in_place_t)(H5D_t *dset, size_t
*write_size /*out*/, bool partial_bound, void **chunk /*in,out*/, void
*udata);
```

The same as *H5SC_chunk_encode_t* but does not preserve chunk buffer, encoding is performed in-place. Must free all other data used.

```
typedef herr_t (*H5SC_chunk_evict_t)(H5D_t *dset, void *chunk, void
*udata);
```

Frees chunk and all memory referenced by it. Optional, if not present *free()* is simply used.

```
typedef herr_t (*H5SC_chunk_insert_t)(H5D_t *dset, size_t count, const
hsize_t *scaled[] /*in*/, haddr_t *addr[] /*in,out*/, hsize_t
old_disk_size[], hsize_t new_disk_size[], void *chunk[] /*in*/, void
*udata[]);
```

Inserts (or reinserts) count chunks into the chunk index if necessary. Old address and size (if any) of the chunks on disk are passed as `addr` and `old_disk_size`, the new size is passed in as `new_disk_size`. This function resizes and reallocates on disk if necessary, returning the address of the chunks on disk in `*addr`.

```
typedef herr_t (*H5SC_chunk_selection_read_t)(H5D_t *dset, const H5S_t
*file_space_in, bool partial_bound, void *chunk /*in*/, H5S_t
**file_space_out /*out*/, bool *select_possible /*out*/, bool
*require_values /*out*/, void *udata);
```

Called when the chunk cache wants to read data directly from the disk to the user buffer via selection I/O. If not possible due to compression, etc, returns `*select_possible=false`. Otherwise transforms the file space if necessary to describe the selection in the on disk format (returns transformed space in `file_space_out`). If no transformation is necessary, leaves `*file_space_out` as NULL, chunk may be passed as NULL, and may also be an in-cache chunk that contains only information about defined values. If chunk is passed as NULL and the callback requires a chunk to be passed with (at least) the defined values selection, this callback shall return `*require_values=true` and `*file_space_out=NULL`. Optional, if not present, chunk I/O is only performed on entire chunks or with vector I/O. The H5SC code checks for type conversion before calling this. `partial_bound` is true if the on-disk chunk was encoded with `partial_bound` set to true. If the dataset reported `partial_bound_chunks_different_encoding` as false, the setting of `partial_bound` is undefined.

```
typedef herr_t (*H5SC_chunk_vector_read_t)(H5D_t *dset, haddr_t addr,
const H5S_t *file_space_in, bool partial_bound, void *chunk /*in*/,
size_t *vec_count /*out*/, haddr_t **offsets /*out*/, size_t **sizes
/*out*/, bool *vector_possible /*out*/, bool *require_values /*out*/,
void *udata);
```

Provides a vector description for directly reading selected portions of an on-disk chunk representation. `file_space_in` describes the selection in logical chunk coordinates and `addr` supplies the chunk's on-disk base address. If direct vector I/O is not possible, such as when decoding or filtering is required, the callback returns `*vector_possible = false`. Otherwise, `*offsets` and `*sizes` describe absolute file ranges and `*vec_count` gives the number of ranges. If defined-value state is required before the vector can be determined, the callback returns `*require_values = true` and does not return a usable vector.

The callback is optional. Structured chunks currently implement this callback, but it is not yet integrated into the normal SCC read path and is presently exercised through dedicated callback tests.

```
typedef herr_t (*H5SC_chunk_selection_write_t)(H5D_t *dset, const
H5S_t *file_space_in, bool partial_bound, void *chunk /*in*/, H5S_t
```

```
*file_space_out /*out*/, bool *select_possible /*out*/, bool
*require_values /*out*/, void *udata);
```

Called when the SCC attempts to perform direct selection I/O from the user buffer to the on-disk chunk representation without first materializing the entire decoded chunk in cache. `file_space_in` describes the requested selection in logical chunk coordinates. If the requested operation cannot be represented as direct selection I/O—for example, because layout-specific encoding or filtering requires full chunk materialization—the callback sets `select_possible` to false. When selection I/O is possible, the callback may transform the logical chunk selection into a selection describing the corresponding on-disk representation. If no transformation is required, no replacement selection need be returned. `chunk` may be `NULL` or may reference a decoded object containing at least defined-value information. If the callback cannot determine the direct-I/O selection without defined-value state that is not currently available, it sets `require_values` to true and does not return a usable direct-I/O selection. `partial_bound` identifies a chunk encoded as a partial-edge chunk when the layout reports that partial-boundary chunks use different encoding.

This callback is optional. The current structured-chunk layout does not implement `selection_write`; structured-chunk writes currently proceed through SCC materialization and gather processing instead.

```
typedef herr_t (*H5SC_chunk_vector_write_t)(H5D_t *dset, haddr_t addr,
const H5S_t *file_space_in, bool partial_bound, void *chunk /*in*/,
size_t *vec_count /*out*/, haddr_t **offsets /*out*/, size_t **sizes
/*out*/, bool *vector_possible /*out*/, bool *require_values /*out*/,
void *udata);
```

Provides a vector description for directly writing selected portions of an on-disk chunk representation. `file_space_in` describes the selection in logical chunk coordinates and `addr` supplies the chunk's on-disk base address. If direct vector I/O is not possible, such as when encoding or filtering is required, the callback returns `*vector_possible = false`. Otherwise, `*offsets` and `*sizes` describe absolute file ranges and `*vec_count` gives the number of ranges. If defined-value state is required before the vector can be determined, the callback returns `*require_values = true` and does not return a usable vector.

The callback is optional. Structured chunks currently implement this callback, but it is not yet integrated into the normal SCC write path and is presently exercised through dedicated callback tests.

```
typedef herr_t (*H5SC_chunk_scatter_mem_t)(H5D_dset_io_info_t
*dset_info, H5D_io_type_info_t *io_type_info, const H5S_t *mem_space,
const H5S_t *file_space, const void *chunk, void *udata);
```

Scatters the elements selected from the decoded chunk representation into the user memory buffer described by `dset_info`. `file_space` describes the selected values in logical chunk coordinates, with an extent matching the chunk dimensions. `mem_space` describes the corresponding destination selection within the complete user memory dataspace. The callback performs any layout-specific translation required between the cached chunk representation and the user's memory representation,

including datatype conversion where appropriate. This callback is optional when the layout's decoded chunk representation is directly compatible with the normal SCC memory-transfer path. A layout that stores variable-length data or otherwise requires layout-specific interpretation of the resident representation must provide this callback. That is more focused and consistent with what `scatter_mem` actually does.

```
typedef herr_t (*H5SC_chunk_gather_mem_t)(H5D_dset_io_info_t
*dset_info, H5D_io_type_info_t *io_type_info, const H5S_t *mem_space,
const H5S_t *file_space, size_t *nbytes /*in,out*/, size_t *alloc_size
/*in,out*/, size_t *alloc_size_total /*in,out*/, void *chunk, void
*udata);
```

Gathers the selected elements from the user memory buffer described by `dset_info` into the decoded resident chunk representation. `file_space` describes the target selection in logical chunk coordinates, while `mem_space` describes the corresponding source selection within the complete user memory dataspace. The callback performs any layout-specific translation and datatype conversion required while defining or replacing the selected values in the chunk. `nbytes` and `alloc_size` describe the used and allocated sizes associated with the primary chunk representation. `alloc_size_total` describes the total resident allocation associated with all buffers owned by the decoded chunk representation and must be updated if the gather operation changes that total. SCC uses this information to reconcile the chunk's actual resident allocation after the write. This callback is optional when the cached chunk representation is directly compatible with the normal SCC memory-transfer path. A layout that stores variable-length data or otherwise requires layout-specific modification of the decoded representation must provide this callback.

```
typedef herr_t (*H5SC_chunk_fill_t)(H5D_dset_io_info_t *dset_info,
H5D_io_type_info_t *io_type_info, H5S_t *space, size_t *nbytes
/*in,out*/, size_t *alloc_size /*in,out*/, size_t *alloc_size_total
/*in,out*/, void *chunk, void *udata);
```

Propagates the dataset fill value into the elements selected by `space` within the decoded resident chunk representation. The selection is expressed in logical chunk coordinates and has an extent matching the chunk dimensions. The callback performs any required layout-specific representation handling and datatype conversion while applying the fill value. `nbytes` and `alloc_size` describe the used and allocated sizes associated with the primary chunk representation. `alloc_size_total` describes the total allocation associated with all resident buffers for the chunk and must be updated if applying the fill value changes that allocation. This callback is optional when the SCC can apply the fill value directly to the layout's resident representation. A layout that requires special handling, including variable-length data, must provide the callback. When `erase_values` is not provided by a layout, the SCC may use `fill` as the fallback mechanism for erase semantics when writing the fill value is equivalent to making the selected values logically erased for that layout.

```
typedef herr_t (*H5SC_chunk_defined_values_t)(H5D_t *dset, const H5S_t
*selection, void *chunk, H5S_t **defined_values /*out*/, void *udata);
```

Queries which values are currently defined within the supplied logical-chunk selection. `selection` describes the region to query and may represent the entire logical chunk through `H5S_ALL`. On success, `defined_values` receives a dataspace selection describing the subset of the requested region that is currently defined. The returned selection is expressed in logical chunk coordinates. This callback is optional. If it is not provided, the SCC treats all values in an allocated chunk as defined. For resident chunks, the callback operates on the current decoded representation and therefore reflects dirty or otherwise unflushed in-memory defined-value state. The current structured-chunk implementation provides this callback and uses it for `H5SC_get_defined()`, erase processing, and related sparse-state operations.

```
typedef herr_t (*H5SC_chunk_erase_values_t)(H5D_t *dset, const H5S_t
*selection, size_t *nbytes /*in,out*/, size_t *alloc_size /*in,out*/,
void *chunk, bool *delete_chunk /*out*/, void *udata);
```

Removes the values selected by `selection` from the decoded chunk's set of logically defined elements. The selection is expressed in logical chunk coordinates. `nbytes` and `alloc_size` describe the current used and allocated sizes associated with the resident representation and must be updated if the erase operation changes those quantities. If erasing the requested values leaves the chunk with no defined elements and the layout determines that the chunk should no longer exist, the callback sets `delete_chunk` to true. The SCC then performs immediate chunk deletion: the persistent chunk/index entry is removed through `H5SC_delete_chunk_t`, the resident representation is released through the normal eviction/teardown path, and the corresponding `H5SC_chunk_t` is removed from SCC tracking. If `delete_chunk` remains false, the modified chunk continues to exist and the caller is responsible for reconciling its resident allocation and dirty/persistent state as required by the operation. This callback is optional. When it is not supplied, the SCC may fall back to applying the fill value through `H5SC_chunk_fill_t` when that behavior provides the layout's required erase semantics.

```
typedef herr_t (*H5SC_chunk_evict_values_t)(H5D_t *dset, size_t
*nbytes /*in,out*/, size_t *alloc_size /*in,out*/, void *chunk, void
*udata);
```

Frees the data values in the cached chunk and memory used by them (but does not reallocate – see `H5SC_chunk_condense_t`), but leaves the defined values intact. Optional, if not present the entire chunk will be evicted.

```
typedef herr_t (*H5SC_layout_query)(H5D_t *dset, hsize_t *chunk_dims,
bool *encode_decode_necessary, bool
*partial_bound_chunks_different_encoding);
```

Queries data about the dataset from the layout client. The callback shall set the chunk dimensions in the `chunk_dims` array (the number of dimensions is the same as the rank of the dataset), whether encoding and decoding is necessary for chunks between cache and disk, and shall set whether chunks that are partially outside the bounds of the dataset are encoded differently (for example, they may not

have filters applied). If `*partial_bound_chunks_different_encoding` is set to `true`, then chunks whose partial bound state changes will be re-encoded and re-inserted as necessary after the dataset extent changes to ensure they are encoded appropriately.

```
typedef herr_t (*H5SC_delete_chunk_t)(H5D_t *dset, const hsize_t
*scaled /*in*/, haddr_t addr, hsize_t disk_size);
```

Removes the chunk from the index and deletes it on disk. Only called if a chunk goes out of scope due to `H5Dset_extent()` or if `H5SC_chunk_erase_values_t` returns `*delete_chunk == true`.

7.1.1 Callback struct

The final callback structure for each layout class is therefore:

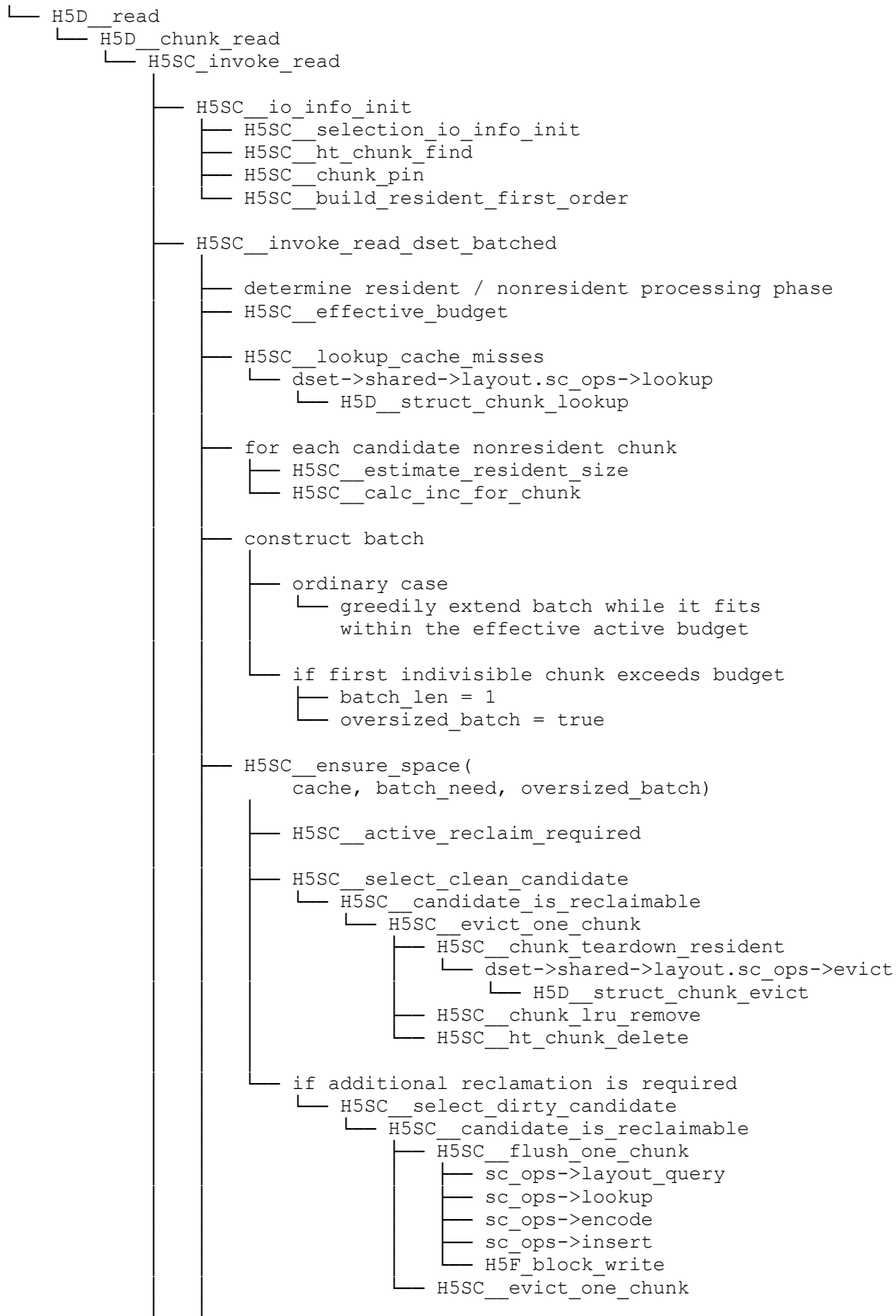
```
const H5SC_layout_ops_t H5SC_LOPS_STRUCT_CHUNK[1] = {{
    H5D__struct_chunk_lookup,          /* lookup */
    H5D__struct_chunk_decode,          /* decode */
    H5D__struct_chunk_decode_defined_values, /* decode_defined_values */
    H5D__struct_chunk_new_chunk,       /* new_chunk */
    H5D__struct_chunk_condense,        /* condense */
    H5D__struct_chunk_encode,          /* encode */
    H5D__struct_chunk_encode_in_place, /* encode_in_place */
    H5D__struct_chunk_evict,           /* evict */
    H5D__struct_chunk_insert,          /* insert */
    NULL,                              /* selection_read */
    H5D__struct_chunk_vector_read,     /* vector_read */
    NULL,                              /* selection_write */
    H5D__struct_chunk_vector_write,    /* vector_write */
    H5D__struct_chunk_scatter_mem,     /* scatter_mem */
    H5D__struct_chunk_gather_mem,      /* gather_mem */
    H5D__struct_chunk_fill,            /* fill */
    H5D__struct_chunk_defined_values,  /* defined_values */
    H5D__struct_chunk_erase_values,    /* erase_values */
    H5D__struct_chunk_evict_values,    /* evict_values */
    H5D__struct_chunk_layout_query,    /* layout_query */
    H5D__struct_chunk_delete_chunk     /* delete_chunk */
}};
```

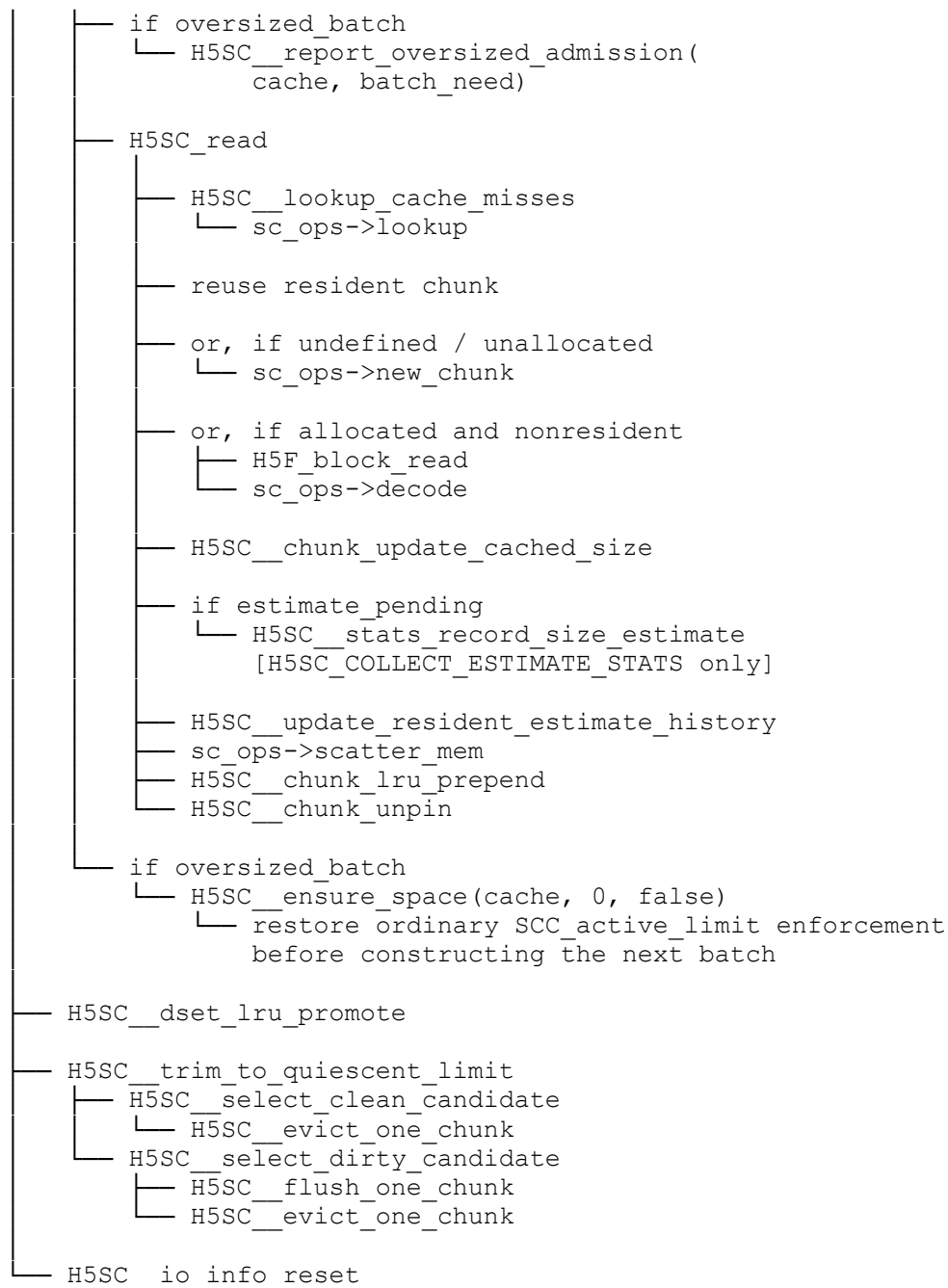
8 Call Tree Diagrams

The following call trees provide a high-level view of the major execution paths through the Shared Chunk Cache and the surrounding HDF5 subsystems. The diagrams emphasize the SCC functions, layout callbacks, cache-management operations, and supporting HDF5 routines that are most relevant to understanding each operation; they are not intended to enumerate every helper function or error-handling path. Where execution is conditional, the diagrams show the principal alternatives that may be exercised depending on cache residency, on-disk chunk state, available cache space, and dataset layout requirements. Together, these call trees illustrate how the higher-level HDF5 dataset operations described earlier in this document are mapped onto SCC initialization, batching, chunk materialization, cache accounting, flushing, eviction, and layout-specific processing.

8.1. Reading data through `H5Dread()`:

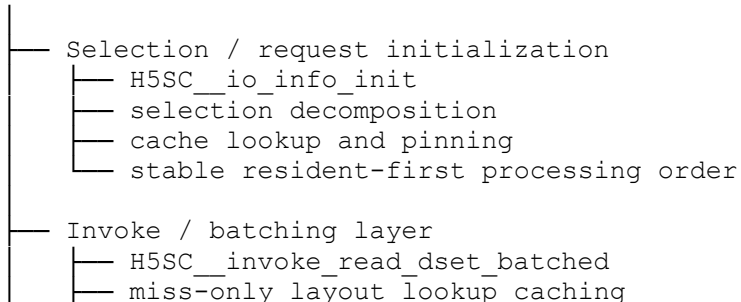
`H5Dread`

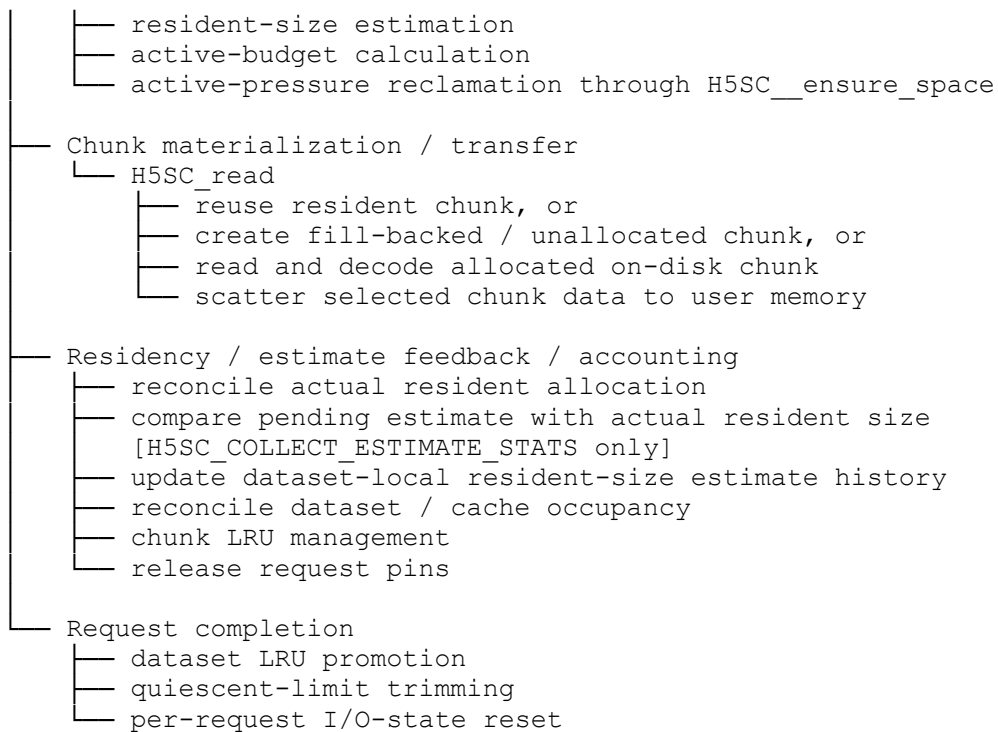




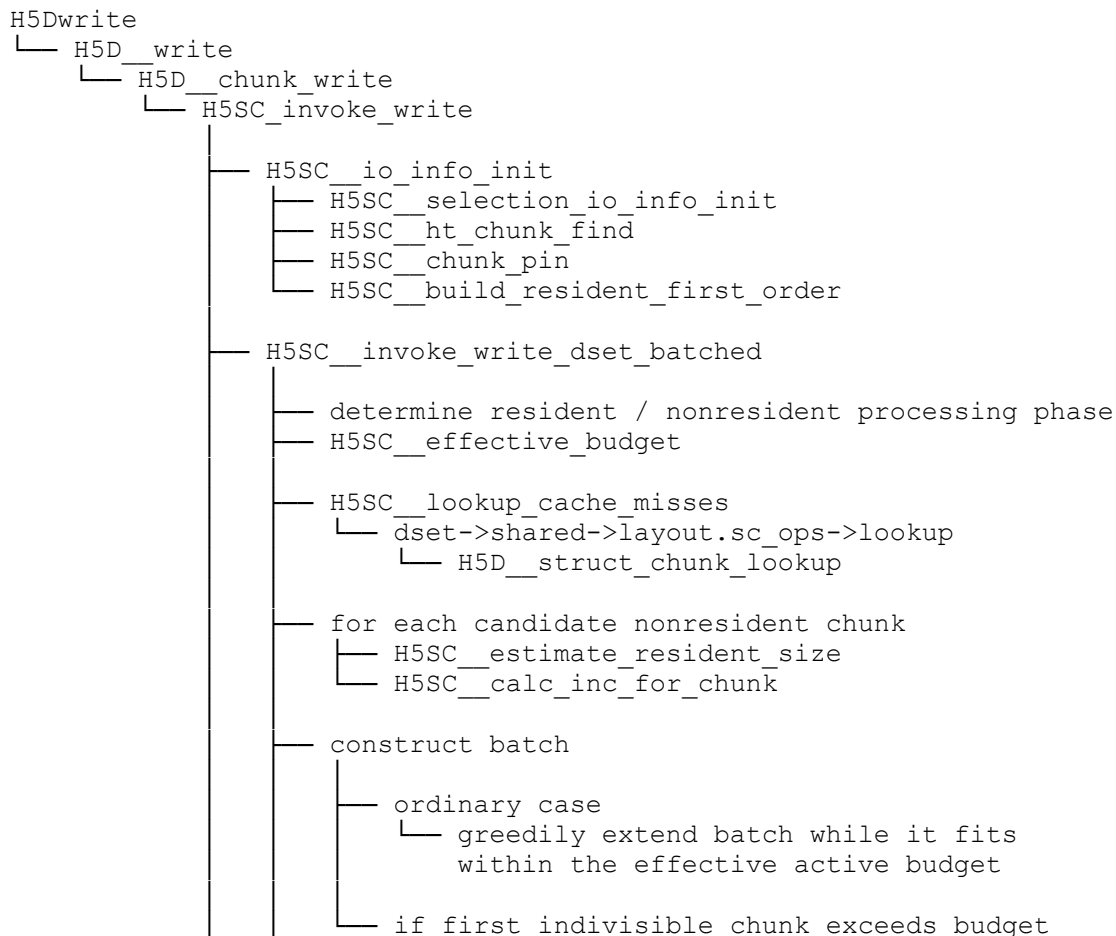
8.1.1 Data Read Call Tree by Subsystem:

Read Path





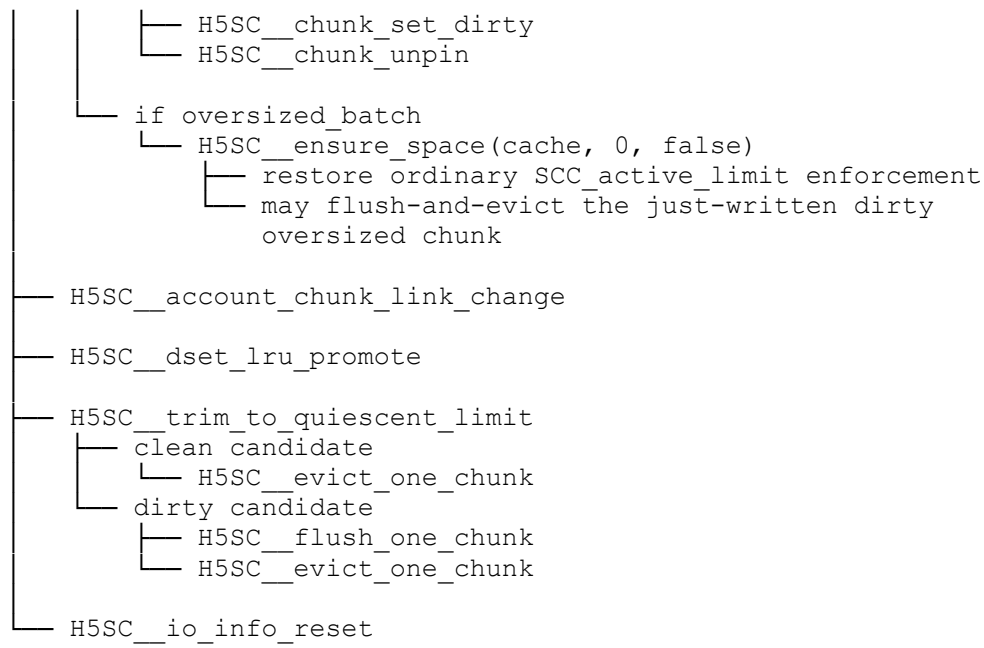
8.2. Writing data through H5Dwrite():



```

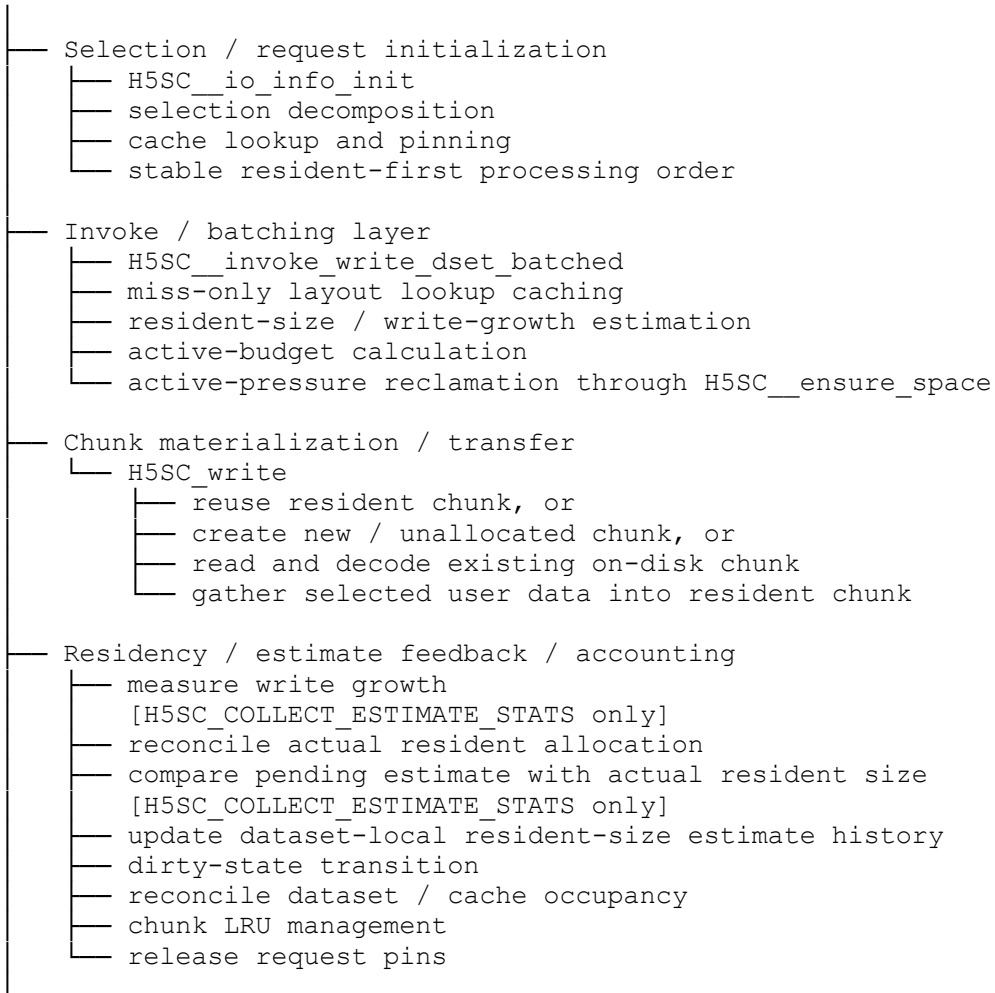
└─ batch_len = 1
└─ oversized_batch = true
H5SC__ensure_space(
  cache, batch_need, oversized_batch)
└─ H5SC__active_reclaim_required
└─ H5SC__select_clean_candidate
    └─ H5SC__candidate_is_reclaimable
        └─ H5SC__evict_one_chunk
            └─ H5SC__chunk_tearardown_resident
                └─ sc_ops->evict
            └─ H5SC__chunk_lru_remove
            └─ H5SC__ht_chunk_delete
└─ if additional reclamation is required
    └─ H5SC__select_dirty_candidate
        └─ H5SC__candidate_is_reclaimable
            └─ H5SC__flush_one_chunk
                └─ sc_ops->layout_query
                └─ sc_ops->lookup
                └─ sc_ops->encode
                └─ sc_ops->insert
                └─ H5F_block_write
            └─ H5SC__evict_one_chunk
if oversized_batch
└─ H5SC__report_oversized_admission(
  cache, batch_need)
H5SC_write
└─ H5SC__lookup_cache_misses
    └─ sc_ops->lookup
└─ sc_ops->layout_query
└─ reuse resident chunk
└─ or, if new / unallocated
    └─ sc_ops->new_chunk
└─ or, if existing and nonresident
    └─ H5F_block_read
    └─ sc_ops->decode
└─ sc_ops->gather_mem
    └─ record write-growth statistics
        [H5SC_COLLECT_ESTIMATE_STATS only]
└─ H5SC__chunk_update_cached_size
└─ if estimate_pending
    └─ H5SC__stats_record_size_estimate
        [H5SC_COLLECT_ESTIMATE_STATS only]
└─ H5SC__update_resident_estimate_history

```



8.2.1 Data Write Call Tree by Subsystem:

Write Path



- └─ Request completion
 - └─ dataset LRU promotion
 - └─ quiescent-limit trimming
 - └─ per-request I/O-state reset

8.3. Supporting Initialization and Dataset Setup

Dataset creation path

- └─ H5SC_dset_create_header
 - └─ initialize H5SC_dset_header_t
 - └─ set min_dset_size from SCC_quiescent_limit policy
 - └─ allocate H5SC_io_info_t

8.4. Eviction and Flush Subsystems

SCC Reclamation / Flush Policies

- └─ 1. Active-pressure reclamation
 - Trigger:
 - H5SC__invoke_{read,write}_dset_batched
 - └─ additional resident bytes must be admitted
 - └─ determine current effective budget
 - └─ H5SC__effective_budget
 - └─ Admission mode
 - └─ Ordinary batch
 - └─ requested incremental bytes fit within normal active-limit admission policy
 - └─ Oversized single-chunk batch
 - └─ first indivisible candidate exceeds current effective active budget
 - └─ batch_len = 1
 - └─ oversized_batch = true
 - └─ H5SC__report_oversized_admission(...) [after successful space preparation]
 - └─ H5SC__ensure_space(bytes_needed, allow_oversized_single_chunk)
 - └─ determine actual admission mode
 - └─ oversized = allow_oversized_single_chunk && bytes_needed > H5SC__effective_budget(cache)
 - └─ ordinary admission
 - └─ verify sufficient reclaimable capacity before mutating cache state
 - └─ H5SC__active_reclaim_required(..., oversized = false)
 - └─ continue until SCC_quiescent_size + bytes_needed <= SCC_active_limit
 - └─ oversized single-chunk admission
 - └─ H5SC__active_reclaim_required(..., oversized = true)

```

    ..., oversized = true)
    └─ continue until
        H5SC__cached_active_reclaimable(cache) == 0
└─ scan datasets from global dataset LRU tail
    └─ Phase 1: clean reclamation
        └─ H5SC__select_clean_candidate(
            ..., H5SC_RECLAIM_ACTIVE, ...)
            └─ H5SC__candidate_is_reclaimable
                └─ H5SC__evict_one_chunk
                    └─ H5SC__chunk_tearardown_resident
                        └─ sc_ops->evict
                            └─ H5D__struct_chunk_evict
                    └─ H5SC__chunk_lru_remove
                    └─ H5SC__ht_chunk_delete
            └─ Phase 2: dirty reclamation
                └─ H5SC__select_dirty_candidate(
                    ..., H5SC_RECLAIM_ACTIVE, ...)
                    └─ H5SC__candidate_is_reclaimable
                        └─ H5SC__flush_one_chunk
                            └─ sc_ops->layout_query
                            └─ sc_ops->lookup
                            └─ sc_ops->encode
                                └─ H5D__struct_chunk_encode
                            └─ sc_ops->insert
                                └─ H5D__struct_chunk_insert
                            └─ H5F_block_write
                        └─ H5SC__evict_one_chunk
Oversized-batch completion:
└─ H5SC_{read,write} processes exactly one oversized chunk
    └─ chunk is unpinned
        └─ H5SC__ensure_space(cache, 0, false)
            └─ restore ordinary SCC_active_limit enforcement
                before subsequent batching

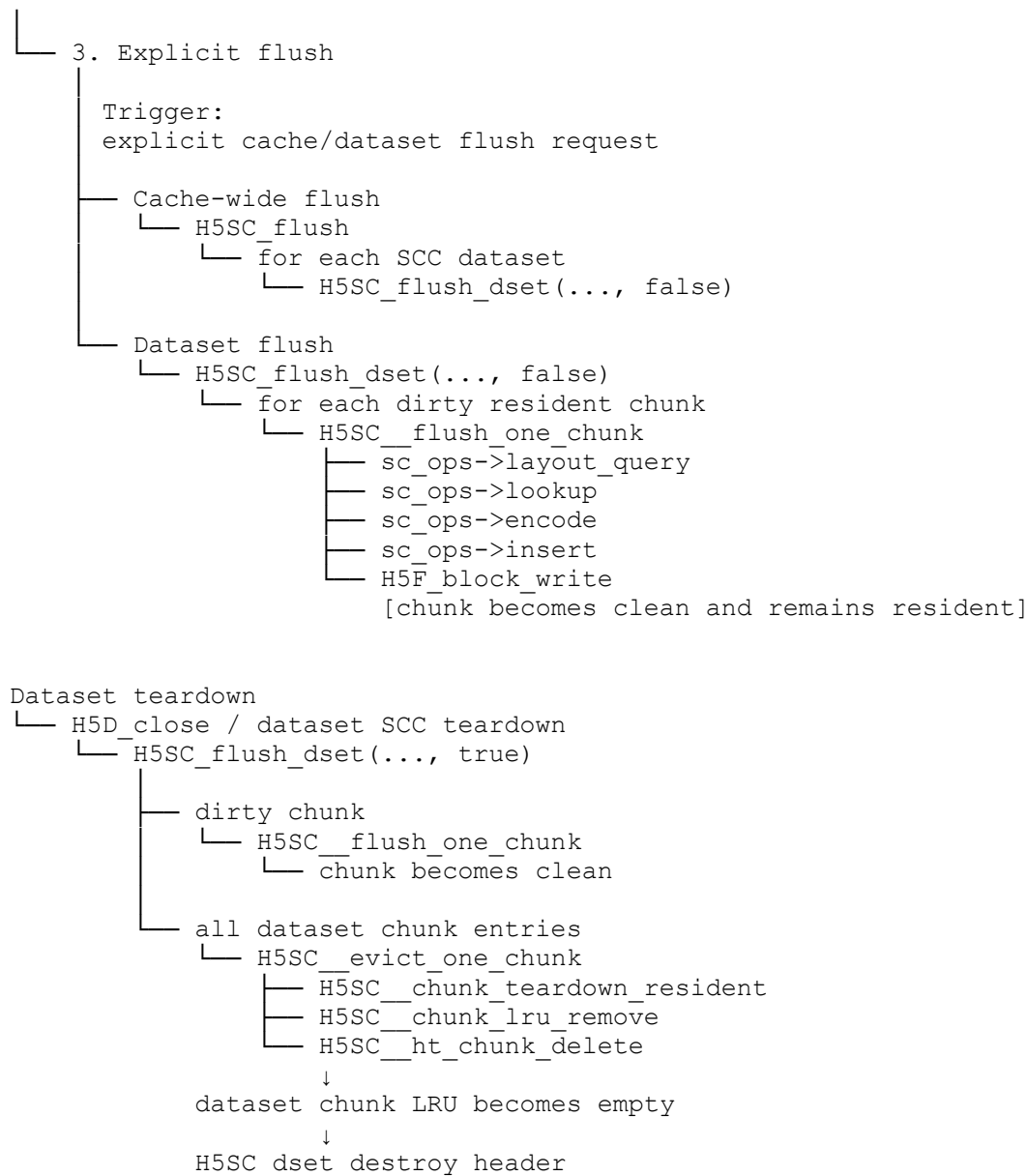
```

2. Quiescent-retention trimming

```

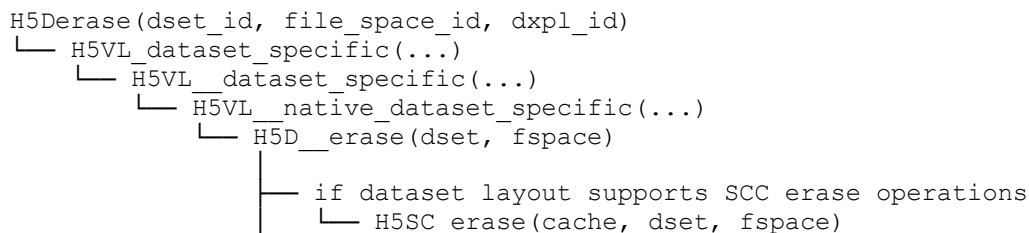
Trigger:
H5SC_invoke_read / H5SC_invoke_write completion
└─ SCC_quiescent_size > SCC_quiescent_limit
└─ H5SC__trim_to_quiescent_limit
    └─ scan datasets from global dataset LRU tail
        └─ H5SC__select_clean_candidate(
            ..., H5SC_RECLAIM QUIESCENT, ...)
            └─ H5SC__candidate_is_reclaimable
                └─ H5SC__evict_one_chunk
        └─ if additional trimming is required
            └─ H5SC__select_dirty_candidate(
                ..., H5SC_RECLAIM QUIESCENT, ...)
                └─ H5SC__candidate_is_reclaimable
                    └─ H5SC__flush_one_chunk
                    └─ H5SC__evict_one_chunk

```



Active-pressure reclamation and quiescent trimming share the same underlying clean/dirty candidate and chunk flush/eviction helpers, but apply different eligibility policies. Active-pressure reclamation may reduce a dataset below `min_dset_size` when required to admit an I/O request, while quiescent trimming treats `min_dset_size` as a dataset-retention target.

8.5. H5Derase() Functionality



```

    H5SC__erase_io_info_init(...)
    └─ H5SC__selection_io_info_init(...)
        └─ decompose file selection into
            chunk-local erase selections
    └─ H5SC__ht_chunk_find(...)
    └─ create SCC shell entry when necessary
    └─ H5SC__chunk_pin(...)

    for each selected chunk
    └─ if chunk has no resident decoded object
        └─ sc_ops->lookup(...)
            └─ H5D__struct_chunk_lookup(...)

        └─ if no on-disk chunk exists
            └─ H5SC__chunk_unpin(...)
                └─ skip undefined sparse region

        └─ else allocated on-disk chunk
            └─ H5F_block_read(...)
            └─ sc_ops->decode(...)
                └─ H5D__struct_chunk_decode(...)

    └─ determine defined elements, when required
        └─ sc_ops->defined_values(...)
            └─ H5D__struct_chunk_defined_values(...)

    └─ apply erase to resident representation
        └─ sc_ops->erase_values(...)
            └─ H5D__struct_chunk_erase_values(...)

    └─ reconcile resident allocation
        └─ H5SC__chunk_update_cached_size(...)

    └─ if erase_values returns delete_chunk == true
        └─ sc_ops->delete_chunk(...)
            └─ H5D__struct_chunk_delete_chunk(...)
        └─ H5SC__chunk_lru_remove(...)
        └─ H5SC__account_chunk_link_change(...)
        └─ H5SC__chunk_tear_down_resident(...)
            └─ sc_ops->evict(...)
                └─ H5D__struct_chunk_evict(...)
        └─ H5SC__ht_chunk_delete(...)
        └─ free H5SC__chunk_t

    └─ else chunk survives
        └─ H5SC__chunk_update_cached_size(...)
        └─ H5SC__update_resident_estimate_history(...)
        └─ H5SC__chunk_set_dirty(..., true)
        └─ H5SC__chunk_unpin(...)

    H5SC__io_info_reset(...)

else
    └─ error: dataset layout does not support erase semantics

```

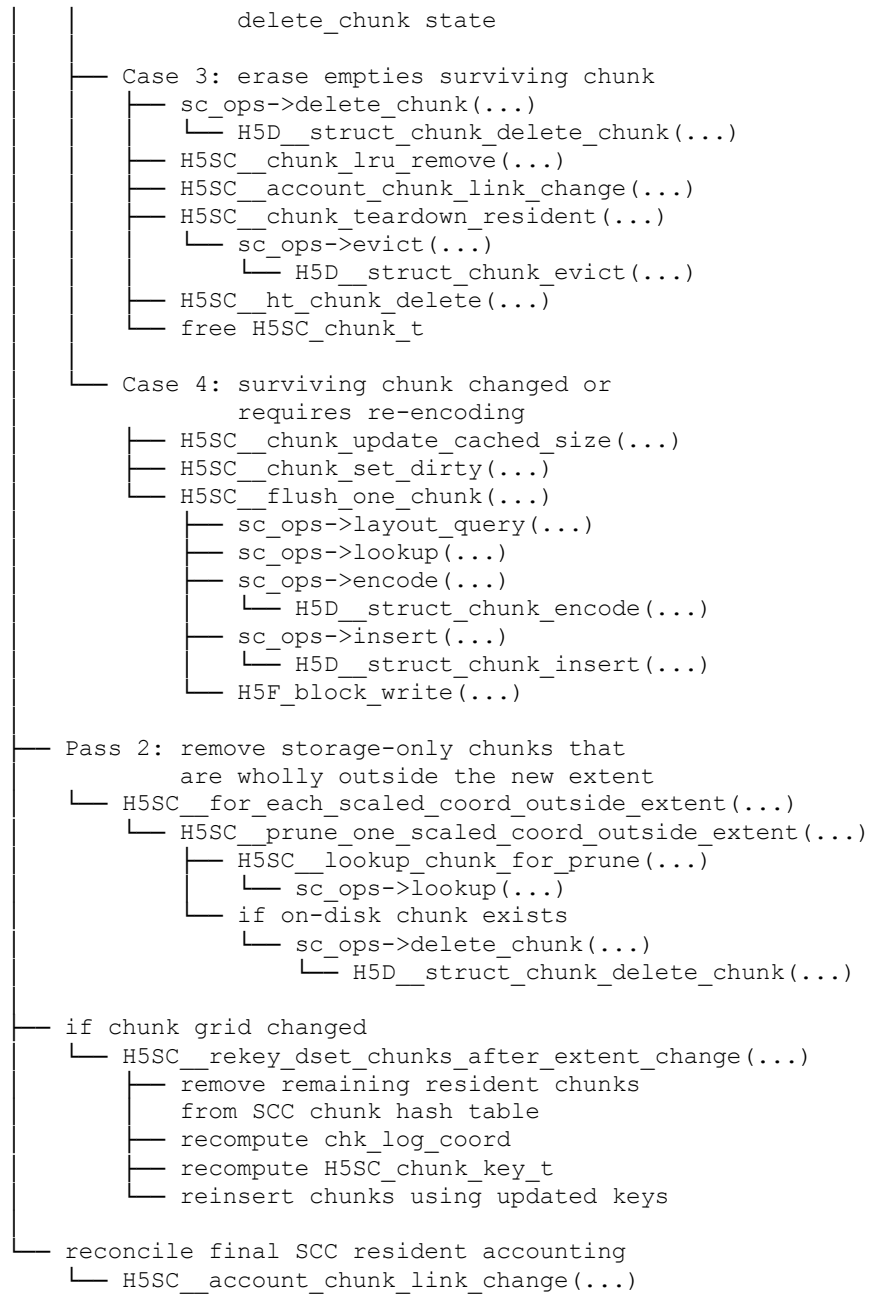
8.6. H5Dset_extent Functionality

H5Dset_extent(dset_id, size)

```

└─ H5D_set_extent_api_common(...)
  └─ H5D_set_extent(dset, size)
    └─ save old_dims
    └─ update dataspace extent
    └─ update layout / chunk-indexing metadata
    └─ if dset->shared->layout.sc_ops exists
      └─ H5SC_set_extent_notify(cache, dset, old_dims)
        └─ read updated extent into ext_dims
        └─ determine whether dimensions shrank and/or expanded
        └─ update dset->shared->curr_dims
        └─ if structured-chunk layout
          └─ H5D_struct_chunk_set_info(dset)
            └─ refresh structured-chunk indexing metadata
        └─ if extent changed and structured storage is allocated
          └─ H5SC_prune_by_extent(cache, dset, old_dims)
            └─ query chunk geometry / encoding policy
              └─ sc_ops->layout_query(...)
            └─ compute old_nchunks / new_nchunks
            └─ determine shrink / growth / chunk-grid change
            └─ if growth only
              └─ if chunk grid changed
                └─ H5SC_rekey_dset_chunks_after_extent_change(...)
                  └─ remove resident dataset chunks
                    from SCC chunk hash table
                  └─ H5SC_compute_logical_chunk_index(...)
                  └─ H5SC_compute_chunk_key(...)
                  └─ reinsert chunks using updated keys
            └─ if shrink
              └─ Pass 1: traverse resident SCC chunks
                └─ classify each resident chunk
                  └─ H5SC_chunk_outside_extent(...)
                  └─ H5SC_chunk_needs_erase(...)
                  └─ determine partial-boundary
                    encoding transition when applicable
                └─ Case 1: chunk fully outside new extent
                  └─ sc_ops->delete_chunk(...)
                    └─ H5D_struct_chunk_delete_chunk(...)
                  └─ H5SC_chunk_lru_remove(...)
                  └─ H5SC_account_chunk_link_change(...)
                  └─ H5SC_chunk_tear_down_resident(...)
                    └─ sc_ops->evict(...)
                      └─ H5D_struct_chunk_evict(...)
                  └─ H5SC_ht_chunk_delete(...)
                  └─ free H5SC_chunk_t
                └─ Case 2: surviving chunk overlaps
                  newly invalid extent region
                  └─ H5SC_erase_chunk_invalid_extent_region(...)
                    └─ determine newly invalid elements
                    └─ construct chunk-local erase selection
                    └─ sc_ops->erase_values(...)
                      └─ H5D_struct_chunk_erase_values(...)
                    └─ return updated resident size /

```



9 Test Coverage Summary

The current H5SC test suite, provided in `./hdf5/test/shared_chunk_cache.c`, provides broad coverage across both the low-level SCC data structures and the higher-level structured-chunk I/O paths. Storage-specific testing is provided in `./hdf5/test/struct_chunk_storage.c`. At the internal-component level, the suite exercises logical chunk index computation, chunk-key construction, bit interleaving/deinterleaving³, dataset and chunk hash tables, dataset-level and per-dataset chunk LRUs,

³ Deinterleaving is not currently used with in the functionality of the H5SC module.

accounting invariants, list splice helpers, lookup helpers, and lifecycle/state-tracking behavior. The logical-index tests validate chunk-grid computations across multiple ranks, including explicit verification of `nchunks[]`, `down[]` multiplier generation, scaled-coordinate bounds checking, and C-order linear chunk index computation against independent reference implementations.

The chunk-key tests provide deterministic and randomized validation of the interleaved 128-bit key generation logic used by SCC chunk identity tracking. These tests verify both individual bit placement and full encode/decode round-trip correctness using randomized fuzz-style input generation. Internal DLL and LRU tests exercise prepend/remove behavior for both dataset-header LRUs and per-dataset chunk LRUs while explicitly validating list ordering, length counters, and resident-byte accounting updates.

The suite also includes lifecycle-oriented regression tests that validate structure integrity and SCC operation tagging across allocation, hash insertion, LRU insertion, flush processing, and lookup revalidation. These tests verify preservation of magic values, pointer identity, and operation-state tracking fields as SCC objects transition through the cache-management pipeline. Collectively, these low-level tests provide strong coverage of SCC bookkeeping correctness, list integrity, hash-table behavior, accounting consistency, key generation, and structural state propagation before higher-level dataset I/O paths are exercised.

At the structured-chunk dataset level, the suite covers the major SCC operational paths through a combination of deterministic correctness tests and large integration-style stress tests. Basic 1D and 2D tests validate single-chunk processing, `H5S_ALL` reads, explicit hyperslab selections, multi-dataset isolation, reopen persistence, and structured chunk dataset creation. The FAPL configuration path is exercised through SCC cache configuration APIs, and multiple tests intentionally use aggressive SCC limits to force eviction, batching, quiescent trimming, and cross-dataset cache turnover.

Extent and erase coverage is substantial. The suite exercises `H5Derase()` over full chunks, partial chunks, undefined chunks, rewritten chunks, and mixed overwrite/erase patterns across multiple ranks. Extent tests cover shrink pruning, storage-only pruning, shrink/regrow cycles, reopen persistence, one and two extensible dimensions, and 3D datasets with multiple changing dimensions. The shrink-prune tests explicitly validate that whole chunks outside the new extent are removed while surviving chunks retain correct data and SCC accounting state. These tests collectively validate synchronization between SCC cache state, structured chunk storage state, and visible dataset contents during complex extent transitions.

Filter coverage is also extensive. `GZIP/DEFLATE` paths are exercised through fixed and selection sections, sparse point-like hyperslab updates, `H5S_ALL` I/O, multidimensional datasets, erase flows, reopen verification, and mixed filter stacks including `SHUFFLE + GZIP` and `FLETCHER32 + GZIP`. `SZIP` coverage now includes `NN/EC` coding modes, multiple pixels-per-block configurations, sparse selections, reopen persistence, erase, shrink/regrow, and mixed-section filtering scenarios where selection metadata uses `GZIP` while fixed payload data uses `SZIP`. The tests also intentionally document current limitations, such as unsupported `SZIP` application to selection metadata sections.

The broadest end-to-end validation is the mixed-rank eviction stress suite. This test exercises seven datasets simultaneously, spanning ranks 1D through 5D and combining extensible and non-extensible datasets, filtered and unfiltered datasets, `H5S_ALL` I/O, sparse updates, erase, shrink/regrow, reopen verification, `SZIP` filtering, mixed-section filtering, and intentionally constrained SCC limits designed to

force aggressive eviction and dataset turnover. The resulting workload provides meaningful integration coverage for SCC accounting, eviction policy, dataset retention rules, and multi-dataset cache interactions under sustained pressure.

Recent SCC testing additionally validates clean/dirty reclaimability accounting, pin/unpin transitions, active versus quiescent reclamation policy, quiescent trimming, resident-first batching, resident-size estimation behavior, flush-without-eviction semantics, flush-and-evict teardown, and cache-accounting invariants across read, write, erase, extent-change, and forced-eviction paths.

10 Multi-threaded support

This section outlines modifications to support multi-thread operations in SCC and external support code required for multi-thread operation of the shared chunk cache.

10.1. Multi-threaded operations in SCC

The following tasks will be performed to support multi-threading in SCC:

1. Switch to multi-thread safe data structures
 - a. Use lock free hash table instead of `UThash` for the chunk and dataset hash tables,
 - b. Replace the doubly linked lists used for the per data set chunk LRUs and the dataset LRU with a lock free equivalent TBD.
2. Modify existing structures to use atomic values where appropriate -- i.e. lengths of and sizes of lists, etc.
3. Code modifications to support the above.
4. While not immediately necessary, for efficiency, we should support concurrent decoding of chunks that have already been read from disk, and encoding of chunks that are about to be flushed and potentially evicted. We need to implement a thread pool to support this functionality and modify the lower-level callbacks as needed to ensure thread safety.

10.2. Additional code for supporting multi-thread operations in SCC

This sub-section outlines external support code required for multi-thread operation of the shared chunk cache.

Serialization module: When I/O requests are serviced in parallel, we must ensure that only overlapping reads are executed concurrently. Overlapping writes, and any intervening overlapping reads must be serialized. The serialization module examines the stream of incoming I/O requests, and delays passing them to the shared chunk cache as necessary to ensure this. When changing a dataset's extent, we drain all current I/O requests and then marks the dataset as locked until the resizing operation is completed.

Resource allocation module: Large I/O requests can be many times the size of the shared chunk cache. Such I/O requests will require exclusive access to the shared chunk cache if we are to avoid thrashing. More generally, we must estimate I/O request resource requirements and only allow

concurrent requests into the shared chunk cache if it has sufficient resources to serve the requests without thrashing. The resource allocation module exists to enforce this.

Multi-thread support in the VFD layer: Concurrent service of multiple I/O requests in the shared chunk cache requires the ability to support concurrent I/O requests in the VFD layer.

Other considerations: Pending multi-thread support in the chunk indices and the metadata cache, we may need to add additional serialization to I/O requests to avoid corruption caused by interleaved modifications to the chunk indexes. This shouldn't be an issue with concurrent reads, however, concurrent writes to the same dataset have potential issues, as do any reads of the dataset concurrent with a write.

The exact potential issues here require further investigation, and it may be that there are no issues not solved by the global mutex.

11 Acknowledgement

This work is supported by the U.S. Department of Energy, Office of Science under award number DE-SC0023583 for Phase II SBIR project “Supporting Sparse Data in HDF5”

12 Appendix A: SCC Version 0 Implementation Description

The Shared Chunk Cache, Version 0 (V0), is a minimally functional representation of how structured chunk data will be processed through the cache. This initial version focuses on demonstrating the connection between the high-level API (through `H5D_write()`) and the low-level callbacks (which are invoked by the SCC). To maintain a simplistic, verifiable output, a single chunk within a single dataset was initially the sole focus of this development, with the end goal being supporting I/O on multiple chunks within a single dataset. To illustrate proper cache functionality, structured chunk-formatted data would first need to be written to a file and then read back. In V0, the cache simply acts as a pass-through for the data buffer between the high-level API and the low-level callbacks, effectively simulating the action of caching the data (i.e. in this instance is assumed that the cache can only “hold” a single chunk; any additional operations involving the cache would require eviction). This approach serves two primary purposes. First, it was necessary to test both the high-level and low-level code integration, which up to this point, had remained untested (by necessity). Second, the V0 cache would provide the developer further exposure to essential internal HDF5 components that would further inform this redesign. With these goals in mind, the V0 development initially focused on creating, writing, and reading a 1-dimensional dataset with five elements and a single chunk of the same size. For simplicity, the chunk would contain non-fill values at indexes 1, 2, and 3 to provide a quick, consistent point of reference.

To facilitate testing the high- and low-level code integration, the write process was the first to be focused on, through the development of the `H5SC_write()` function. Using chunk information derived from first calling `H5SC_io_info_init()`, two key tasks are done; first the caching action for a dirty chunk is emulated, followed by a simulation of the eviction process. Upon calling `H5SC_write()`, `H5SC_io_info_init()` is called, which processes selections for each chunk in the I/O request. Once

this preliminary chunk information is collected, the chunk is looked up on disk (which, by assumption, will result in a failure to find the chunk). Since the chunk is not found on disk, it should be fully overwritten, which leads to invoking the new chunk callback to initialize a new chunk. After the new chunk is created, the gather memory callback is called to collect the chunk data into a single buffer. Next, the "eviction" process begins. First, the chunk is encoded to ensure that it is transitioned from the in-memory format into the on-disk format. Then, the chunk is inserted into the file indexing using the insert callback function. Finally, the encoded chunk buffer is written to disk using the `H5F_block_write()` function. On-disk data verification was done using the Unix octal dump (`od`) tool.

After the write process was debugged and verified, the next step was to read the data back into a user buffer through developing the `H5SC_read()` function. Similar to the procedure used to write the initial chunk, `H5SC__io_info_init()` is used to collect necessary information about the requested chunk. Using this information, the chunk is looked up (and this time around, it is expected to be present). If the chunk is found on disk, the chunk read process is initiated. Using the information collected with the lookup callback, `H5F_block_read()` is then invoked to load the on-disk formatted chunk into a buffer. This buffer is then run through the decode callback, which translates the data into the in-memory format. Finally, this buffer is scattered into the available memory using the scatter memory callback.

Once this process was completed for a single chunk in a single dataset, it was then extended to support writes and reads on a single dataset with multiple chunks. During this process, the callbacks were tested further and adjusted to ensure proper functionality when an I/O request necessitates processing multiple chunks.

13 References

1. John Mainzer, Elena Pourmal, “RFC: New Requirements for HDF5 Chunk Cache”, https://github.com/LifeboatLLC/SparseHDF5/blob/main/design_docs/RFC-HDF5-Chunk-Cache-Req-2023-12-18.pdf.
2. Neil Fortner, John Mainzer, Vailin Choi, Clay Carper, “RFC: Shared Chunk Cache Internal API”, https://github.com/LifeboatLLC/SparseHDF5/blob/main/design_docs/Shared_Chunk_Cache_API.v11.pdf
3. Hanson, Troy. “Uthash: A Hash Table for c Structures.” *Github.io*, 2012, troydhanson.github.io/uthash/.

Revision History

<i>July 25, 2025:</i>	Version 0 sent for internal review.
<i>July 31, 2025:</i>	Version 0.1 sent for internal review.
<i>August 27, 2025</i>	Version 0.2-0.8 copyediting; 0.6 version added new section of multi-threaded updates.
<i>April 8, 2026</i>	Version 0.9 updated to reflect Version 1 development and changes; began integrating the content of other design documents for cohesion and ease of access.
<i>April 22, 2026</i>	Version 0.10 provided further updates to reflect Version 1 development and changes.
<i>May 19, 2026</i>	Version 0.11 includes updates to reflect Version 1 development, including filter support and testing coverage.
<i>August 30, 2026</i>	Version 0.12 includes updates to reflect code the completed code review for Version 1.